

BUILD YOUR OWN

Matter over Thread Smart Home Devices

A practical DIY guide — from installing the SDK to shipping real devices

nRF54L15 · nRF Connect SDK v3.3.0 · Zephyr · 16 products, 29 units deployed

Ver : 09112026-0.1

Who this is for

- You want to build your OWN smart-home devices — not buy them
- You want them to work in Apple Home AND Home Assistant, natively
- You have some embedded experience, but Matter/Thread is new to you
- You would rather not repeat a year of other people's mistakes

What makes this guide different

Everything here was learned by building and deploying real devices in two homes.

The Lessons Learned section is the most valuable part — each entry cost days to find.

What we built

16

products

29

units deployed

2

homes

1

shared codebase

- 6 sensors: contact, temp+humidity+pressure, temp+humidity, lux, gesture, air quality
- 4 plugs & switches: smart plug (CT), smart plug (HLW8032), 3-gang switch, camera switch
- 5 controllers: six-button switch, Hitachi A/C bridge, door lock, fan, garage door
- 1 infrastructure: a mains temp/humidity sensor that also acts as a Thread router
- $6 + 4 + 5 + 1 = 16$, matching the product summary table later in this deck

PART 1

Concepts

What Matter and Thread actually are —
and what they are not.

Why Matter over Thread?

- One device works in Apple Home, Google Home, Alexa AND Home Assistant
 - No cloud, no vendor account, no bridge — devices talk locally
- Thread is a low-power mesh: battery sensors last years, mains devices relay
 - Self-healing — if one node dies, traffic routes around it
- IPv6 end to end: every device is directly addressable
- Open standard — your device is not at the mercy of one vendor's app

Why 'directly addressable' actually matters

- No middleman: the controller talks to the device, with nothing translating
 - Classic ZigBee / 433 MHz kit needs a vendor hub; when it dies, everything dies
- No NAT, no port forwarding — the IPv6 space is big enough for every device
- You can ping6 a specific device and debug with ordinary network tools
- Multicast lets one message address a whole group instead of N unicasts
- Scale is not limited by a hub bottleneck — adding devices adds nodes, not hub load
- Firmware updates, debugging and monitoring go straight to the device

The stack — who does what

Your application	The C/C++ firmware YOU write: reads sensors, drives relays
Matter	Data model, clusters, commissioning, security
Thread	IPv6 mesh routing, network formation
IEEE 802.15.4	2.4 GHz radio, MAC layer
nRF54L15 hardware	SoC, radio, peripherals

Note: all five layers run ON YOUR DEVICE. Apple Home, Google Home, Alexa and Home Assistant are NOT on this diagram — they are the controller at the other end, running on a different machine.

Device roles — this decides everything

Role	Power	Radio	Typical use
FTD (router)	mains	always on	plugs, switches, controllers
MTD / SED	battery	sleeps	contact, temp, lux sensors
ICD (LIT/SIT)	battery	polls parent	Matter's sleepy-device profile
Border Router	mains	always on	HomePod, Apple TV, HA + dongle

Why it matters

A sleepy device only listens when it polls. Every timeout, retry and recovery behaviour differs from a mains device. Most of our hardest bugs came from this.

Fabrics and commissioning

- A FABRIC is one controller's network — Apple Home is one, HA is another
- A device can join several fabrics at once (multi-admin) — ours join both
- COMMISSIONING = the pairing handshake that puts a device into a fabric
 - BLE carries the handshake, then the device joins Thread
 - The controller verifies the device's certificate before trusting it
- Sessions (CASE) live in RAM — they DIE on power loss and must be rebuilt

CASE sessions — what actually carries your data

Matter has two secure channels, used for completely different jobs:

	PASE	CASE
Stands for	Passcode Authenticated	Certificate Authenticated
Used when	only during commissioning	every exchange after that
Authenticated by	the 8-digit passcode (SPAKE2+)	each side's operational cert (NOC)
Carried over	BLE	Thread / IPv6

- CASE is a Sigma1 / Sigma2 / Sigma3 handshake: both sides verify certificates, then derive a symmetric key
- That key exists only in RAM — power loss, reboot or a firmware update destroys it
- After a reboot the device must REBUILD every CASE session before data flows again
 - this is exactly why a sensor takes time to come back after a power cut (Lesson 2)
- CASE resumption can skip the full handshake using the previous session — faster and cheaper

PART 2



Hardware

The silicon and the module —
and why we chose them.

nRF54L15 — the SoC

Feature	Spec	Why it matters
CPU	Arm Cortex-M33 128 MHz	runs Matter + Thread comfortably
Flash (RRAM)	1524 KB	Matter is big; you need the room
RAM	256 KB	Thread + Matter + BLE all resident
Radio	2.4 GHz multiprotocol	BLE and 802.15.4 share one radio
Security	TrustZone, KMU	key storage for DAC private key

The multiprotocol radio is the subtle part: BLE and Thread take turns. Long BLE events starve Thread — that broke commissioning for us more than once.

E73-2G4M08S1F — why we chose it

- The real benefit is high integration in a very small footprint:
 - nRF54L15 + 32 MHz HFXO + 32.768 kHz LFXO, all inside one shield can
 - the DCDC inductor, antenna and matching network are in there too
 - so your carrier only has to handle power and I/O — no RF section at all
- The hardest part of RF layout — impedance control, matching, antenna keep-out — is done for you

Do not assume certification is the benefit

This module carries Chinese certification, but NOT Taiwan's NCC. So in Taiwan its existing approvals do not help — you still have to certify the product yourself. Choose it for integration and size, not to skip certification.

What is shared is the **CONVENTION**, not the board

- The 16 products do NOT share one PCB — each product has its own board
- What they share is one set of GPIO conventions for the features every product has:
 - status indicator LED → P1.10
 - factory-reset button → P1.06
 - the second LED the framework insists on → P1.14
- The payoff: one Board implementation and one debug workflow apply to every product

Only four of them genuinely share a board — temp/humidity, pressure, lux and gesture — because all four are I2C parts with identical wiring. The only difference is which sensor you populate.

Board-common pins — identical on every product

These three pins are defined in the carrier's .dts, not in any product overlay:

Pin	Function	Configuration
P1.06	Factory-reset button (sw0)	PULL_UP ACTIVE_LOW, long-press >= 6 s wipes the fabric
P1.10	Status LED (led0)	ACTIVE_HIGH, lit only while commissioning / long-pressing
P1.14	Fake LED (led1)	ACTIVE_HIGH, not wired – satisfies the framework only

The 'fake LED' on P1.14 exists because Nordic's Matter Board library initialises a second LED unconditionally. The garage door controller needs P1.14 for its reed contact, so it remaps that LED to P0.04.

How long does a CR2032 last, and how do you report it?

- A CR2032 is 220 mAh nominal; ~200 mAh usable after pulse-load and the 2.0 V cutoff
- All six battery products wake every 300 s, deliberately aligned with the ICD slow poll
 - one wake does everything: read sensor, read battery, poll the parent, sleep again
- Sleep current then dominates the life completely — sampling barely registers

Voltage	Reported	What it means
≥ 2900 mV	100%	plateau – voltage barely moves under light load
2700 mV	70%	the knee – energy starts dropping
2400 mV	20%	well into the drop; replace it soon
≤ 2000 mV	0%	cutoff

Piecewise, not linear, on purpose: a linear scale badly over-reports what is left — when it shows 30%, a real CR2032 has about 5%, giving almost no warning.

Two platform quirks in battery reporting

- HomeKit will not show a battery at all without BatPercentRemaining
 - BatChargeLevel alone (OK / Warning / Critical) is ignored entirely by HomeKit
 - Home Assistant is perfectly happy with just BatChargeLevel
- The Power Source cluster must sit on EP0 with EndpointList = {0}
 - on EP1 it works in HA but is invisible in HomeKit
- Add hysteresis to the thresholds (we use 30 mV) or the level flickers at the edges

Note: every battery-life figure in this deck is CALCULATED from datasheet values and the configured intervals — not bench-measured. A CR2032 has high internal resistance, so RF pulses sag the rail and real life is usually noticeably shorter.

The nRF54L15 SAADC pin map (a trap)

Analog input numbers are NOT sequential with port numbers, and only port 1 has them:

```
AIN0 = P1.04    AIN1 = P1.05    AIN2 = P1.06    AIN3 = P1.07  
AIN4 = P1.11    AIN5 = P1.12    AIN6 = P1.13    AIN7 = P1.14
```

We really did walk into this

The smart plug's current transformer sits on P1.04, but the overlay said AIN4. AIN4 is actually P1.11 — a button held at 3.3 V by its internal pull-up. The ADC read a rock-steady 3789 codes, which became a plausible, never-changing 12-14 W.

Battery products need no divider at all: they read the internal NRF_SAADC_VDD directly.

PART 3



The Toolchain

Installing nRF Connect SDK and VS Code —
the part that stops most people.

What is the nRF Connect SDK?

- NCS is not one thing — it is a bundle:
 - Zephyr RTOS — the kernel, drivers, devicetree, Kconfig
 - Nordic drivers and libraries — radio, MPSL, power management
 - Matter (connectedhomeip) — the whole Matter stack, as a module
 - A pinned toolchain — compiler, CMake, Python, west
- 'west' is the meta-tool that ties the repositories together

PIN YOUR VERSION. We use v3.3.0 everywhere. Matter and Zephyr both move fast; mixing versions is the fastest way to lose a weekend.

Setting up VS Code — step by step

1

Install Visual Studio Code

code.visualstudio.com

2

Install the nRF Connect for VS Code Extension Pack

from the Marketplace

3

Open the nRF Connect side panel → Manage toolchains

installs compiler + tools

4

Install SDK v3.3.0 via the same panel

downloads Zephyr, Matter, everything

5

Install SEGGER J-Link

for flashing and RTT logging

6

Install nRF Util (nrfutil)

command-line flashing

Verify the install

Open the nRF Connect terminal (it sets PATH for you) and check:

```
west --version
nrfutil --version
nrfutil device list          # should show your J-Link
arm-zephyr-eabi-gcc --version
```

If 'nrfutil device list' shows 'Supported devices found: 0', the J-Link is not enumerating — replug the USB cable before you debug anything else. We lost time to this repeatedly.

PART 4



Project Anatomy

What every file in a project does.

Folder layout

```
apps/my_product/  
  CMakeLists.txt      which source files to build  
  prj.conf            Kconfig options – the main config file  
  Kconfig             product-specific Kconfig defaults  
  sysbuild.conf       bootloader / multi-image build options  
  pm_static_<board>.yaml flash partition layout  
  VERSION            firmware version (used by DFU)  
  boards/<board>.overlay devicetree: your sensors, pins, buses  
  src/  
    main.cpp          entry point  
    app_task.cpp       Matter init + your app logic  
    zcl_callbacks.cpp  incoming Matter commands land here  
    default_zap/       the Matter data model (see Part 6)  
  prj_production_overlay.conf  release build settings
```

prj.conf — the main config

```
CONFIG_CHIP=y                               enable Matter
CONFIG_CHIP_DEVICE_VENDOR_ID=65524          0xFFF4 (test VID)
CONFIG_CHIP_DEVICE_PRODUCT_ID=32769         0x8001
CONFIG_CHIP_DEVICE_DISCRIMINATOR=0x080     BLE pairing identifier
CONFIG_CHIP_DEVICE_SPAKE2_PASSCODE=<your 8-digit code>
CONFIG_CHIP_DEVICE_TYPE=21                 0x0015 contact sensor
```

Kconfig is hierarchical and full of defaults you did not set. When something behaves oddly, read the GENERATED config — `build/<app>/zephyr/.config` — not `prj.conf`. They are not the same, and the generated one is the truth.

Devicetree overlay — your hardware

```
&i2c30 {  
    status = "okay";  
    clock-frequency = <I2C_BITRATE_STANDARD>;  
  
    sht35: sht3xd@44 {  
        compatible = "sensirion,sht3xd";  
        reg = <0x44>;  
    };  
};
```

- Devicetree describes hardware; Kconfig enables the software for it
- You almost always need BOTH: the node here AND CONFIG_SHT3XD=y in prj.conf

PART 5

Build & Flash

From source to a device you can pair.

Building

```
west build -b e73_battery_sensor/nrf54l15/cpuapp \  
  -d C:/nrf-builds/my_product \  
  -p always \  
  C:/nrf-projects/apps/my_product \  
  -- -DBOARD_ROOT=C:/nrf-projects
```

- -p always = pristine build. Slower, but avoids stale-config mysteries
- -d puts build output OUTSIDE the source tree — keeps the repo clean
- BOARD_ROOT tells west where your custom board definition lives
- We wrap this in build.cmd so nobody has to remember it

Flashing

```
nrftutil device program --firmware merged.hex \  
    --options chip_erase_mode=ERASE_ALL --x-family nrf54l  
nrftutil device reset --x-family nrf54l
```

Erase mode	What survives	When to use
ERASE_ALL	nothing	identity change, clean slate
(default)	settings_storage	firmware update, keeps pairing

ERASE_ALL wipes the Matter fabrics — the device must be re-paired in every ecosystem. Omit it when you only want to update firmware.

Debug vs production builds

Setting	Debug	Production
Logging / RTT	on	off
LTO	off	on (~20% smaller)
RESET_ON_FATAL_ERROR	n – halts forever	y – reboots
Shell	available	removed

The trap

Debug builds HALT on a fault instead of rebooting. A crashed device looks identical to a hung one. In production it would simply have restarted.

PART 6



The Matter Data Model

Endpoints, clusters, and the ZAP tool.

Endpoints and clusters

- ENDPOINT = one logical device. EP0 is always the root (device info, network)
- CLUSTER = a feature. OnOff, BooleanState, TemperatureMeasurement, ...
- ATTRIBUTE = a value inside a cluster. Controllers read and subscribe to these
- DEVICE TYPE = tells the controller what the endpoint IS (0x0015 = contact sensor)

EP0	root	Basic Information, Network Commissioning
EP1	Temp Sensor	TemperatureMeasurement
EP2	Humidity Sensor	RelativeHumidityMeasurement
EP3	Pressure Sensor	PressureMeasurement

ZAP — and the trap that costs everyone a day

- ZAP is the GUI/JSON tool that defines your endpoints and clusters
- It generates C++ into `src/default_zap/zap-generated/`

BYPASS_IDL

The build does NOT regenerate `zap-generated/` from your `.zap` file. Editing the `.zap` changes nothing until you run the generator manually. We shipped a wrong data model this way.

So: after every `.zap` edit, run your regen script.

PART 7

Certificates & Factory Data

The part nobody explains — and the reason
your device will refuse to pair.

The certificate chain

PAA	Product Attestation Authority — the root
PAI	Product Attestation Intermediate
DAC	Device Attestation Certificate — per product, encodes VID+PID

Home Assistant's Matter Server only trusts PAAs it knows. We had to re-root every product under the SDK's standard test PAA — a custom PAA is rejected with 'PAA not found in trust store' and the device simply will not pair.

The Certification Declaration (CD)

- A signed blob saying 'this VID + PID is a real certified product'
- It contains a LIST of product IDs it covers
- On Nordic it lives in the app image as a macro — NOT in factory data

This cost us a full debugging session

We widened the CD to cover more PIDs — then created a new product with a PID outside the OLD copy of the CD that its folder still carried. The device could not be commissioned at all. Every product has its own copy; copies drift.

A pairing code is three things

- DISCRIMINATOR (12 bits) — tells one advertising device from another over BLE
- PASSCODE (8 digits) — the SPAKE2+ shared secret; this is the actual security
- QR / manual code — those two plus VID and PID, encoded together
- Change any of them and the label must be reprinted — a QR is a snapshot

The key idea

The discriminator only NAMES the device; the passcode is the LOCK. So the discriminator must be unique but need not be secret — while the passcode may be shared, but must be treated as a password.

Rule 1 — allocate the discriminator, don't derive it

```
discriminator = (product_code << 7) | unit_seq      // 12 bits total  
                5 bits (1-31)      7 bits (0-127)
```

- Why not hash the SoC ID the way the serial does? Only 12 bits = 4096 values
 - ~10% chance of a collision across 30 units of one product
 - ~78% across 80 devices in one home — completely unusable
- Structured allocation is collision-free BY CONSTRUCTION, not by probability
- Serial derived, discriminator allocated — different needs, different mechanisms

Rule 2 — why a weaker guarantee is fine here

	Duplicate serial	Stale discriminator
Consequence	a ghost device in HA	the QR matches nothing
When you notice	possibly never	immediately, at commissioning
How to fix	clean up the registry	reprint one label
Nature	a silent failure	a self-announcing failure

You can tolerate a failure that announces itself. The things that must be absolutely unique are the ones that go wrong without telling you.

Rule 3 — unit_seq is never typed by a human

- provision.py reads the board's SoC ID first, then decides:
 - a board it has seen before → reuse the discriminator already in units.csv
 - a new board → allocate the next free unit_seq
- Recorded values are used VERBATIM, never recomputed
 - so reflashing can never silently change a board's discriminator
 - which means a printed QR label stays valid for the life of the board
- It also refuses to run if you connect a different product's board

```
python provision.py --product camera_switch  
# units.csv: unit_seq, serial, discriminator, flashed, location, notes
```

Rule 4 — the QR is per-unit, the manual code is not

- The 11-digit manual code carries ONLY discriminator bits 11:8
 - under this layout those bits equal `product_code >> 1`
 - so they are independent of `unit_seq` — identical for every unit
- That asymmetry gives you both things at once:
 - a per-unit QR → target one specific board in a batch
 - a shared manual code → the fallback when a label is lost or unreadable

Verified on camera_switch, 2026-08-28: three units produced three distinct QR payloads and one identical manual code — exactly as predicted.

Rule 5 — one passcode across the portfolio, deliberately

- Every product shares one 8-digit passcode. This is a deliberate choice
 - It is safe because the discriminators are already unique — you cannot commission the wrong device
 - It avoids managing 16 secrets, and 'which code was this one?' never happens
- The Matter spec forbids trivial passcodes. Do not use:
 - 00000000, 11111111 ... 99999999 (any repeated digit)
 - 12345678, 87654321

But note

A shared passcode only suits devices you own and administer yourself. The moment a device goes to someone else, give it its own passcode — and never commit a passcode to a public repository.

Three field lessons about pairing codes

Changing the PID invalidates every printed QR

The QR encodes VID / PID / discriminator / passcode. Touch any one of them and the labels must be reprinted — changing our PIDs forced a regeneration of `factory_data.png`.

We had a real discriminator collision

temphumidity-router was on 0xE93, which was ALSO Door_Controller's. Two devices advertising the same discriminator simply cannot be told apart by a commissioner.

A copied `gen_qr.py` printed another product's number

3-cut-switch's `gen_qr.py` encoded Car_Door's discriminator — copy-paste that was never cleaned up.

Discriminator, passcode, QR

Thing	What it does	Scope
Discriminator	identifies device during BLE pairing	must be unique nearby
Passcode	the 8-digit pairing PIN	can be shared
QR code	encodes VID/PID/discriminator/passcode	printed on the label
Serial number	shown in the app; HA keys devices on it	MUST be unique

Home Assistant keys its device registry on the SERIAL NUMBER ALONE. Two devices sharing one serial become ONE device in HA, with both sets of entities.

PART 8



Device Identity at Scale

The architecture that stopped us
from shipping duplicate devices.

The problem

- Serial number was a compile-time constant — every unit reported the same one
- Home Assistant fused them: ONE device carrying 35 entities from 3 products
- Worse, several DIFFERENT products shipped the SAME constant by copy-paste
 - SixButton and the A/C bridge both said DZNRF54L00103
 - Gesture sensor and air-quality sensor both said DZNRF54L00105

The fix — derive identity from the chip

```
DZ54L-01-B7196C
|      |  |
|      |  +-- FNV-1a hash of the SoC's unique ID (24 bits)
|      +----- product code
+----- vendor + platform
```

- Same firmware image flashes to every unit — each one self-identifies
- Reflashing reproduces the SAME serial, so HA keeps the same device entry
- No counters, no per-unit factory data, no allocation table to lose

One product code drives everything

```
product code -> PID           = 0x8000 + code
              -> DAC prefix    = pNN-
              -> discriminators = (code << 7) | unit   (127 per product)
              -> serial prefix  = DZ54L-NN-
```

- Codes are PERMANENT and never reused — even for retired products
- Collisions become impossible by construction, not by discipline
- A provisioning script reads the chip ID, allocates the unit, records it in CSV

PART 9



The Product Catalogue

16 products, one shared codebase.

Product summary (1 of 2)

Product	Code	PID	Device type	Power	Units
contact_sensor	1	0x8001	0x0015	battery	4
TempHumidity_BMP280	2	0x8002	0x0302	battery	1
Lux_sensor	3	0x8003	0x0106	battery	1
SixButton	4	0x8004	0x000F	battery	1
HitachiAC	5	0x8005	0x0301	mains	3
Gesture_sensor	6	0x8006	0x000F	battery	1
Smart_Plug	8	0x8008	0x010A	mains	2
3-cut-switch	9	0x8009	0x010A	mains	2

Product summary (2 of 2)

Product	Code	PID	Device type	Power	Units
camera_switch	10	0x800A	0x010A	mains	4
ENV_sensor	11	0x800B	0x002C	mains	2
Pulse_Lock	12	0x800C	0x000A	mains	0
fan_control	15	0x800F	0x002B	mains	2
Door_Controller	17	0x8011	0x0202	mains	1
temphumidity-router	18	0x8012	0x0302	mains	1
TempHumidity_SHT35	19	0x8013	0x0302	battery	1
AC_SmartPlug (HLW8032)	20	0x8014	0x010A	mains	1

Each product then gets its own spec page: what it does, how it is wired, its Matter data model, its firmware modules, and what actually went wrong on it.

Contact Sensor

apps/contact_sensor/

Reports open/closed as a magnet approaches or leaves, plus battery level.

Product code	Product ID	Matter device type	Endpoints	Power / Thread role	Bootloader / OTA	Deployed
01	0x8001	0x0015 Contact Sensor	2	Battery / ICD sleepy	MCUboot + BLE DFU	4

Hardware notes	Matter data model	Firmware modules
· Hall sensor → P0.01 · No I2C — the simplest sensing hardware here · SAADC reads internal VDD directly as battery level	· EP1 BooleanState (StateValue) · EP1 PowerSource (battery level / status) · ICD: slow poll 300 s, idle 300 s · ICD: active threshold 15 s, fast poll 500 ms	· battery_monitor.cpp, watchdog.cpp

CR2032 life estimate	Usable ~200 mAh	Wake period 300 s	Avg current ~5.2 µA	Theoretical ~4.3 yr	Realistic 1.5–2.5 yr	Dominant load Event-driven hall IRQ; battery sampled every 300 s
-------------------------	--------------------	----------------------	------------------------	------------------------	-------------------------	---

GPIO map (including the board-common LED and button)			
P0.01	Hall sensor input (ACTIVE_LOW)	P1.06	Factory-reset button (long-press >= 6 s)
gpio0	sense-edge-mask 0x2: wake on P0.01 edge	P1.10	Status LED (lit during commissioning)
SAADC	Battery voltage (internal VDD, no external divider)	P1.14	Fake LED (framework placeholder, unwired)

FIELD NOTE

Lesson 1 (bt_rx stack overflow) and Lesson 2 (six-hour backoff) were both found on this product.

Contact Sensor

Schematic

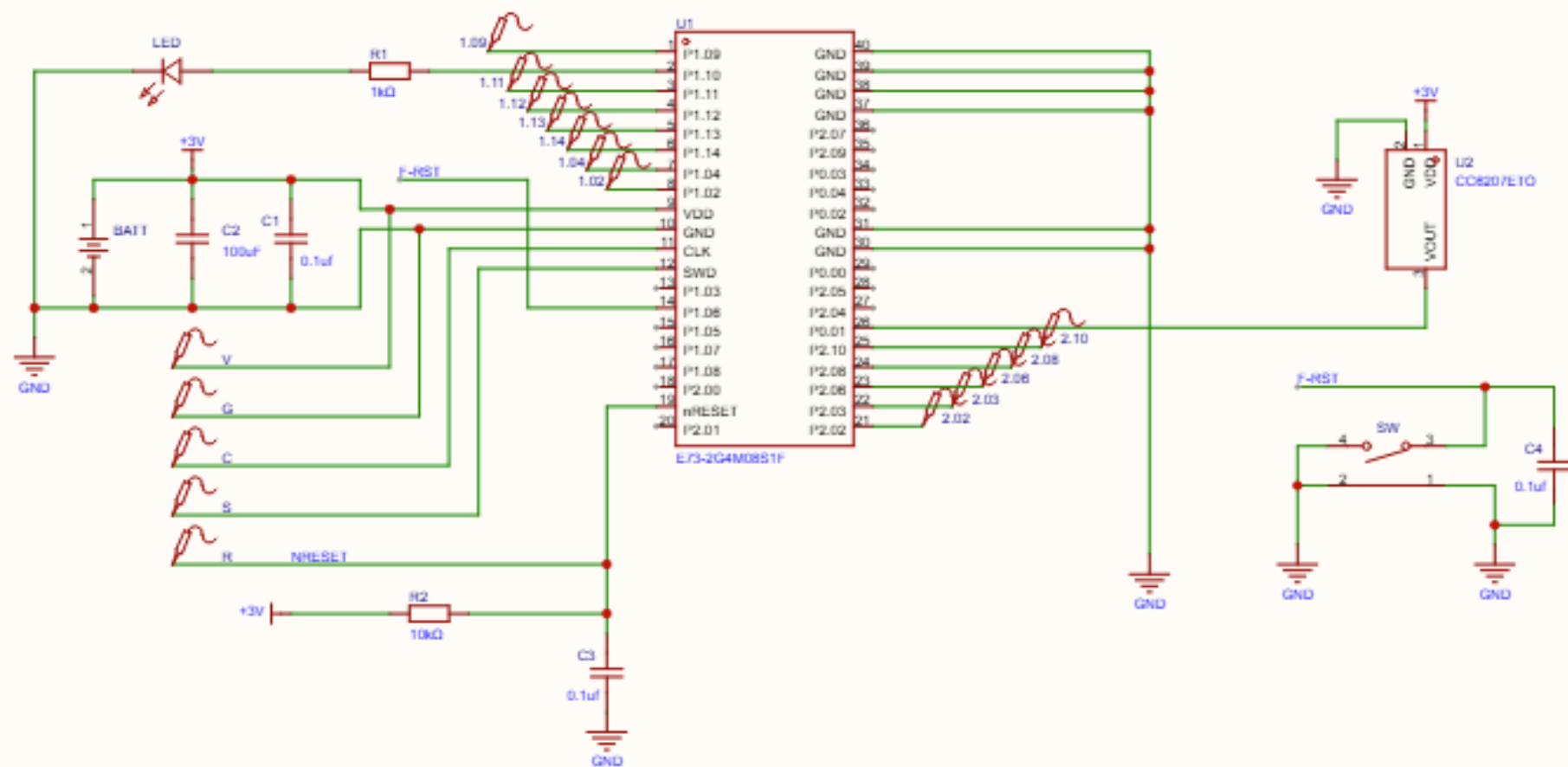
GPIO

P1.10 : LED

P1.14: Fake LED to meet the requirement
of Nordic Matter Board library.

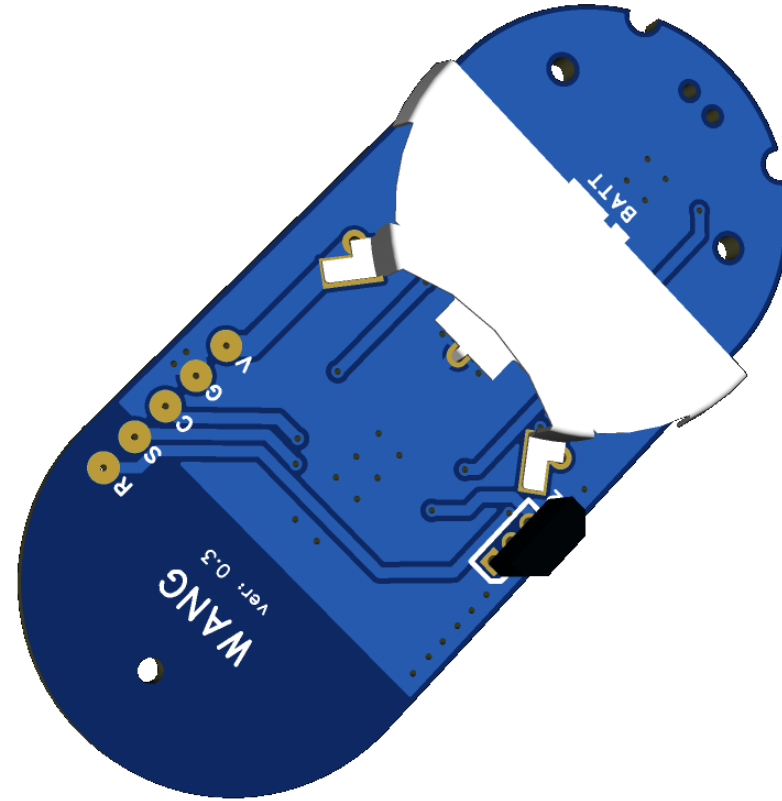
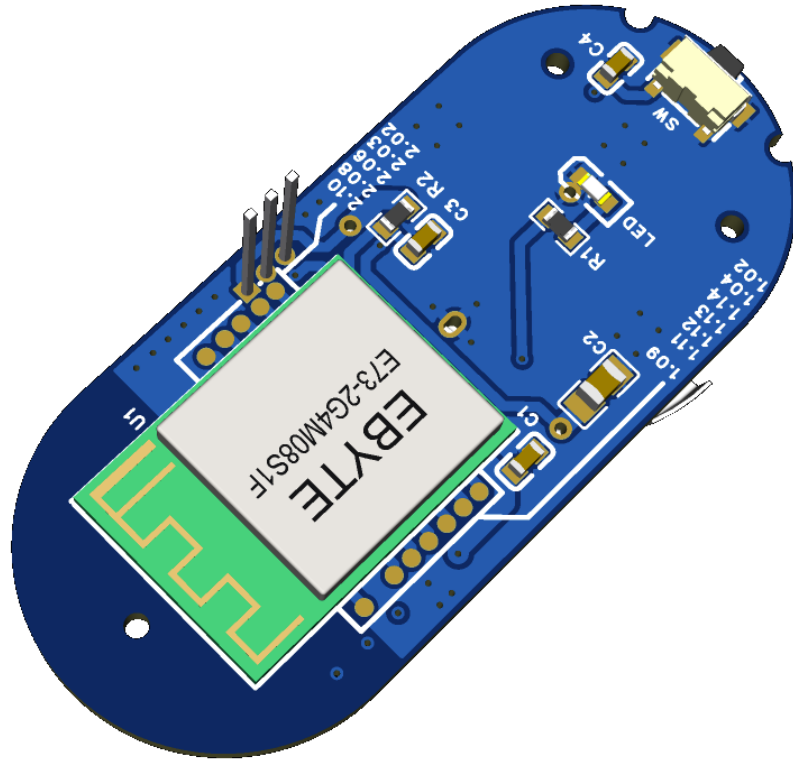
P0.01 :

P1.06 : Factory Reset



Contact Sensor

PCBA-3D



Temp / Humidity / Pressure Sensor

apps/TempHumidity_Sensor_BMP280/

Two sensors on one board reporting temperature, humidity and barometric pressure.

Product code	Product ID	Matter device type	Endpoints	Power / Thread role	Bootloader / OTA	Deployed
02	0x8002	0x0302 Temperature Sensor	4	Battery / ICD sleepy	none (v1, no bootloader)	1

Hardware notes	Matter data model	Firmware modules
· SHT35 @ 0x44 (temperature + humidity) · BMP280 @ 0x76 (via the bosch,bme280 driver) · Both share i2c30, SCL P0.00 / SDA P0.01 · SAADC internal VDD for battery	· EP1 TemperatureMeasurement · EP2 RelativeHumidityMeasurement · EP3 PressureMeasurement · EP1 PowerSource (battery)	· temp_humidity_sensor.cpp, pressure_sensor.cpp · battery_monitor.cpp, watchdog.cpp

CR2032 life estimate	Usable ~200 mAh	Wake period 300 s	Avg current ~5.4 µA	Theoretical ~4.2 yr	Realistic 1.5–2 yr	Dominant load SHT35 + BMP280 read every 300 s, ~10-15 ms each
-------------------------	--------------------	----------------------	------------------------	------------------------	-----------------------	--

GPIO map (including the board-common LED and button)					
P0.00	I2C30 SCL (TWIM30)	SAADC	Battery voltage (internal VDD, no external divider)		
P0.01	I2C30 SDA (TWIM30)	P1.06	Factory-reset button (long-press >= 6 s)		
I2C 0x44	SHT35 temperature + humidity	P1.10	Status LED (lit during commissioning)		
I2C 0x76	BMP280 pressure sensor	P1.14	Fake LED (framework placeholder, unwired)		

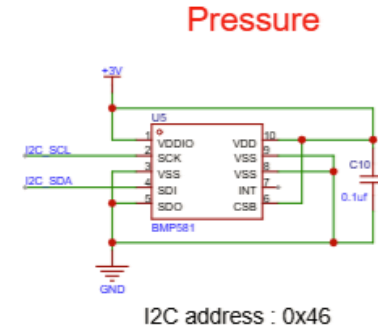
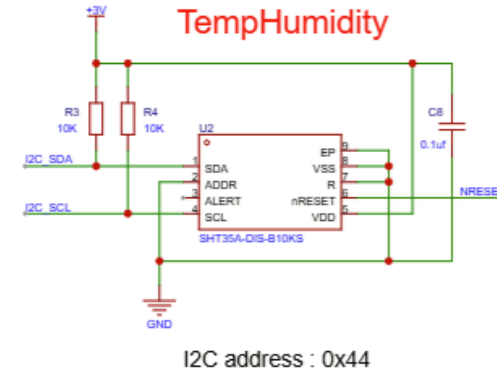
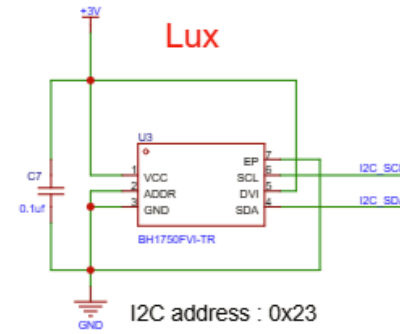
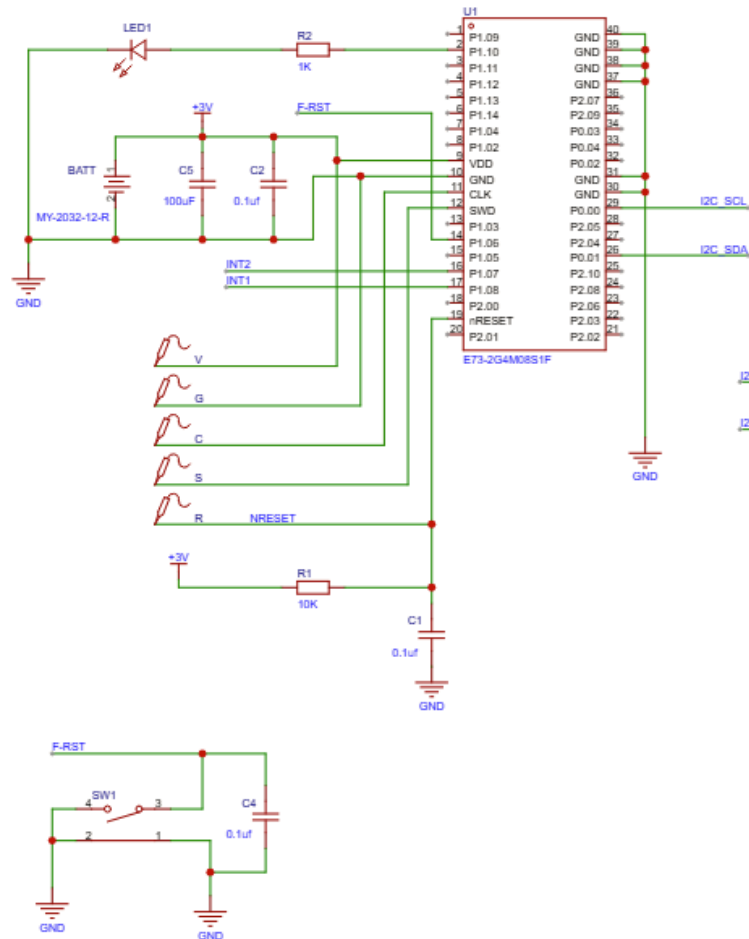
FIELD NOTE

Source of Lesson 7: counterfeit BMP280 parts whose temperature channel always returns zero, corrupting pressure compensation.

Temp / Humidity / Pressure Sensor

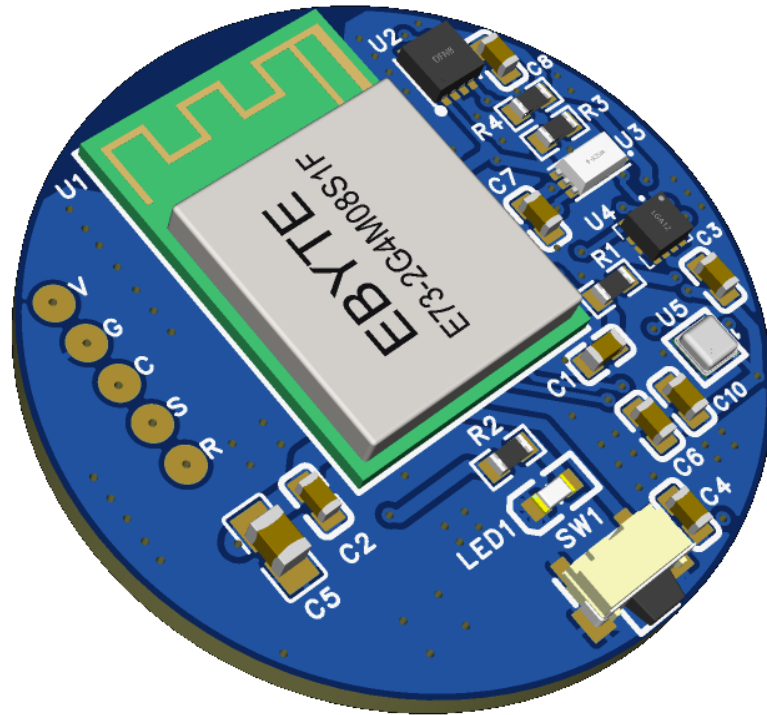
Schematic

GPIO
P1.10 : LED
P1.14 : Fake LED to meet Nordic Matter Board library.
P0.00 : I2C_SCL
P0.01 : I2C_SDA
P1.06 : Factory Reset
P1.07 : INT2
P1.08 : INT1



Temp / Humidity / Pressure Sensor

PCBA-3D



Lux Sensor

apps/Lux_sensor/

Measures ambient illuminance — the trigger for 'turn the lights on at dusk' automations.

Product code	Product ID	Matter device type	Endpoints	Power / Thread role	Bootloader / OTA	Deployed
03	0x8003	0x0106 Light Sensor	2	Battery / ICD sleepy	none (v1, no bootloader)	1

Hardware notes	Matter data model	Firmware modules
- BH1750 @ 0x23 (digital lux meter) - i2c30: SCL P0.00 / SDA P0.01 - SAADC internal VDD for battery	- EP1 IlluminanceMeasurement - EP1 PowerSource (battery) - Same ICD parameters as the contact sensor	lux_sensor.cpp, battery_monitor.cpp

CR2032 life estimate	Usable ~200 mAh	Wake period 300 s	Avg current ~5.5 µA	Theoretical ~4.1 yr	Realistic 1.5–2 yr	Dominant load BH1750 one-shot every 300 s, ~180 ms in H mode
-------------------------	--------------------	----------------------	------------------------	------------------------	-----------------------	---

GPIO map (including the board-common LED and button)					
P0.00	I2C30 SCL (TWIM30)	P1.06	Factory-reset button (long-press >= 6 s)		
P0.01	I2C30 SDA (TWIM30)	P1.10	Status LED (lit during commissioning)		
I2C 0x23	BH1750 digital lux meter	P1.14	Fake LED (framework placeholder, unwired)		
SAADC	Battery voltage (internal VDD, no external divider)				

FIELD NOTE

Apple Home's accessory tile table does not include Light Sensor (0x0106), so iOS display needs a workaround.

Refer to Page 56 and 57

Six-Button Switch

apps/SixButton/

Six independent buttons, each its own Matter switch endpoint, bindable to different scenes.

Product code	Product ID	Matter device type	Endpoints	Power / Thread role	Bootloader / OTA	Deployed
04	0x8004	0x000F Generic Switch	7	Battery / ICD sleepy	none (v1, no bootloader)	1

Hardware notes	Matter data model	Firmware modules
· All six buttons on gpio1 (has GPIOTE, so they can interrupt) · All configured GPIO_PULL_UP GPIO_ACTIVE_LOW · The factory-reset pin P1.06 was chosen for THIS product's routing · SAADC internal VDD for battery	· EP1–EP6 one Switch cluster each · 7 endpoints total (including EP0 root) · EP1 PowerSource (battery)	· six_button.cpp, battery_monitor.cpp

CR2032 life estimate	Usable ~200 mAh	Wake period 300 s	Avg current ~5.2 µA	Theoretical ~4.3 yr	Realistic 1.5–2.5 yr	Dominant load Event-driven button IRQs; battery sampled every 300 s
-------------------------	--------------------	----------------------	------------------------	------------------------	-------------------------	--

GPIO map (including the board-common LED and button)						
P1.05	Button 1 → EP1	P1.08	Button 5 → EP5	P1.06	Factory-reset button (long-press >= 6 s)	
P1.04	Button 2 → EP2	P1.07	Button 6 → EP6	P1.10	Status LED (lit during commissioning)	
P1.09	Button 3 → EP3	全部	All: PULL_UP ACTIVE_LOW	P1.14	Fake LED (framework placeholder, unwired)	
P1.11	Button 4 → EP4	SAADC	Battery voltage (internal VDD, no external divider)			

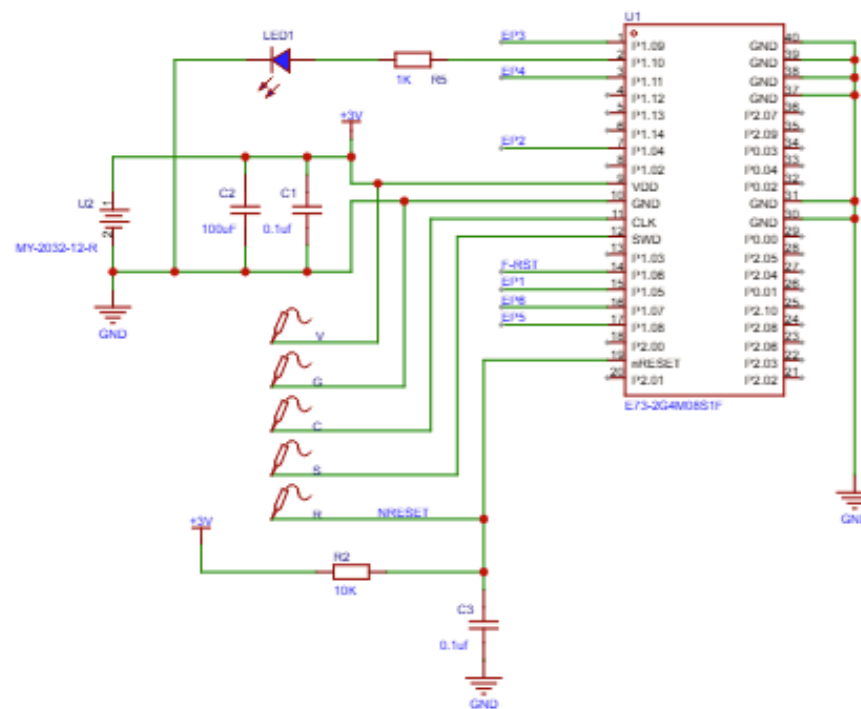
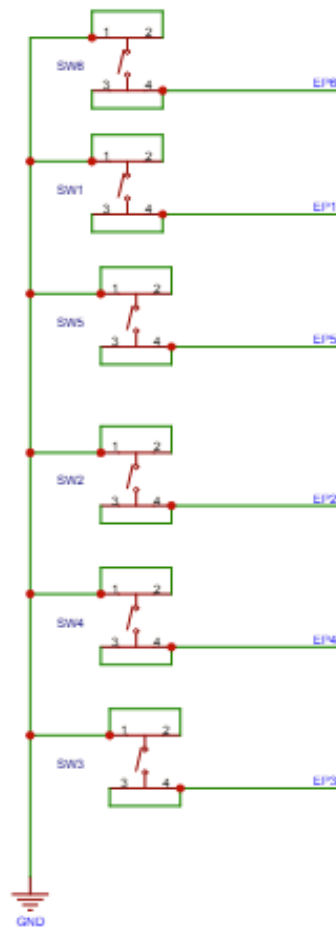
FIELD NOTE

Once shared a hard-coded serial (DZNRF54L00103) with the Hitachi A/C bridge — one of the cases behind Lesson 5.

Six-Button Switch

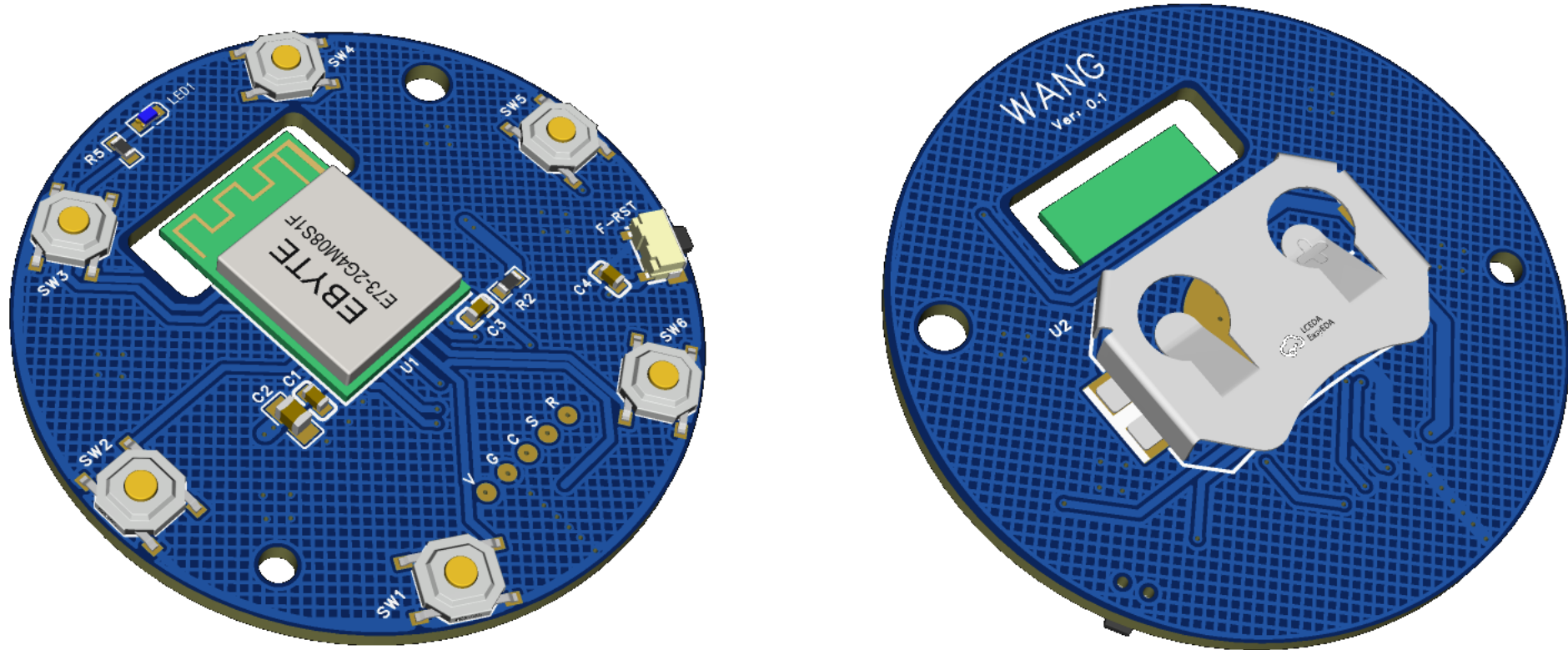
Schematic

GPIO
P1.10: LED
P1.14: Fake LED
P1.05: EP1
P1.04: EP2
P1.09: EP3
P1.11: EP4
P1.08: EP5
P1.07: EP6
P1.06: Factory Reset



Six-Button Switch

PCBA-3D



Hitachi A/C Bridge

apps/HitachiAC/

Drives a Hitachi A/C over its native TaiSEIA serial interface, exposing it as a standard Matter thermostat.

Product code	Product ID	Matter device type	Endpoints	Power / Thread role	Bootloader / OTA	Deployed
05	0x8005	0x0301 Thermostat	2	Mains / Thread router	MCUboot + BLE DFU	3

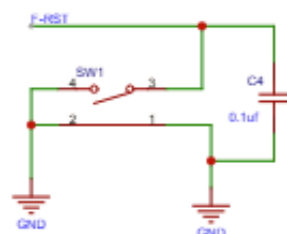
Hardware notes	Matter data model	Firmware modules
· UART20 to the A/C's TaiSEIA interface · RX P1.07 / TX P1.08 — both in the gpio1 power domain · SERIAL20 natively drives port 1, hence the UART lives there · No extra sensors — room temperature comes from the A/C	· EP1 Thermostat (mode, setpoint, local temperature) · EP1 FanControl (fan speed) · Appears as a Thermostat in both Apple Home and HA	· taixia_uart.cpp (protocol I/O) · taixia_climate.cpp (state machine) · temperature_manager.cpp

GPIO map (including the board-common LED and button)			
P1.07	UART20 RX ← A/C unit	P1.10	Status LED (lit during commissioning)
P1.08	UART20 TX → A/C unit	P1.14	Fake LED (framework placeholder, unwired)
P1.06	Factory-reset button (long-press >= 6 s)		

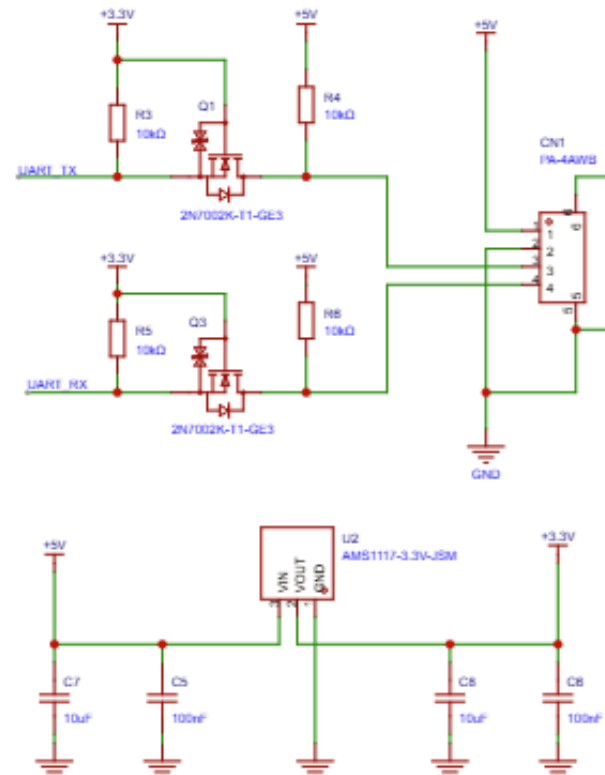
FIELD NOTE

Two older units still carry the old bootloader key — SWD only; OTA would silently succeed and do nothing (Lesson 3).

Schematic

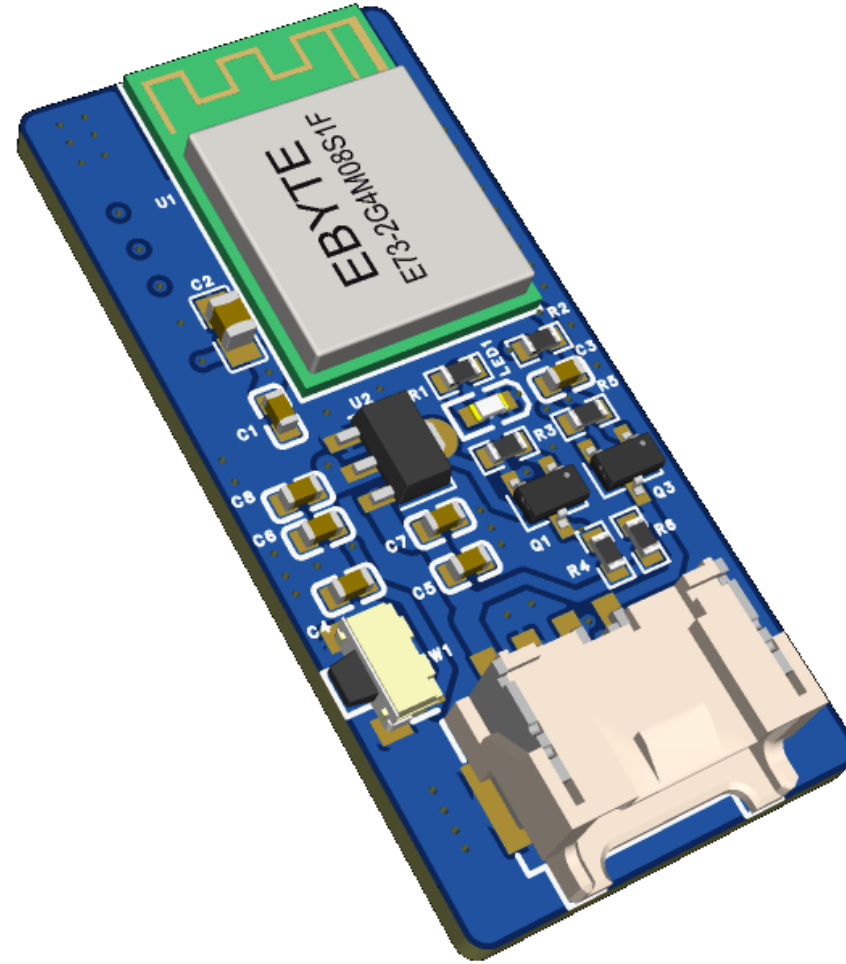


P1.06 : Factory Reset



Hitachi A/C Bridge

PCBA-3D



Gesture Sensor

apps/Gesture_sensor/

Uses an accelerometer to detect taps and orientation changes, turning gestures into Matter switch events.

Product code	Product ID	Matter device type	Endpoints	Power / Thread role	Bootloader / OTA	Deployed
06	0x8006	0x000F Generic Switch	2	Battery / ICD sleepy	none (v1, no bootloader)	1

Hardware notes	Matter data model	Firmware modules
· LIS2DW12 @ 0x18 (3-axis accelerometer) · Interrupt line → P1.08, so it can sleep fully between events · i2c30: SCL P0.00 / SDA P0.01 · SAADC internal VDD for battery	· EP1 Switch (gestures mapped to button events) · EP1 PowerSource (battery)	· gesture.cpp (interrupt-driven gesture detection) · battery_monitor.cpp

CR2032 life estimate	Usable ~200 mAh	Wake period 300 s	Avg current ~6.7 µA	Theoretical ~3.4 yr	Realistic 1–2 yr	Dominant load LIS2DW12 always-on low-power detect, ~1.5 uA - the biggest extra
-------------------------	--------------------	----------------------	------------------------	------------------------	---------------------	---

GPIO map (including the board-common LED and button)					
P0.00	I2C30 SCL (TWIM30)	SAADC	Battery voltage (internal VDD, no external divider)		
P0.01	I2C30 SDA (TWIM30)	P1.06	Factory-reset button (long-press >= 6 s)		
P1.08	LIS2DW12 interrupt IRQ (ACTIVE_HIGH)	P1.10	Status LED (lit during commissioning)		
I2C 0x18	LIS2DW12 3-axis accelerometer	P1.14	Fake LED (framework placeholder, unwired)		

FIELD NOTE

Interrupt-driven and fully asleep between events — the most power-efficient battery design here.

Refer to Page 56 and 57

Smart Plug

apps/Smart_Plug/

Switches mains power remotely or locally, and measures load power with a current transformer.

Product code	Product ID	Matter device type	Endpoints	Power / Thread role	Bootloader / OTA	Deployed
08	0x8008	0x010A On/Off Plug-in Unit	2	Mains / Thread router	MCUboot + BLE DFU	2

Hardware notes	Matter data model	Firmware modules
- Relay → P1.05, push button → P1.07 - CT → P1.04 = SAADC AIN0 (NOT AIN4!) - CT output rides a ~1.5 V bias from the R4/R8 10k/10k divider - P1.05 and P1.07 are both on gpio1, which has GPIOTE	- EP1 OnOff - EP1 ElectricalPowerMeasurement (power / current) - EP1 Identify	- relay_control.cpp, metering.cpp - ElectricalPowerMeasurementDelegate.cpp - ble_config_service.cpp, watchdog.cpp

GPIO map (including the board-common LED and button)			
P1.05	Relay (ACTIVE_HIGH)	P1.06	Factory-reset button (long-press >= 6 s)
P1.07	Push button (PULL_UP / ACTIVE_LOW)	P1.10	Status LED (lit during commissioning)
P1.04	Current transformer CT → SAADC AIN0	P1.14	Fake LED (framework placeholder, unwired)
ADC	12-bit, gain 1/4, DC bias ~1.5 V subtracted		

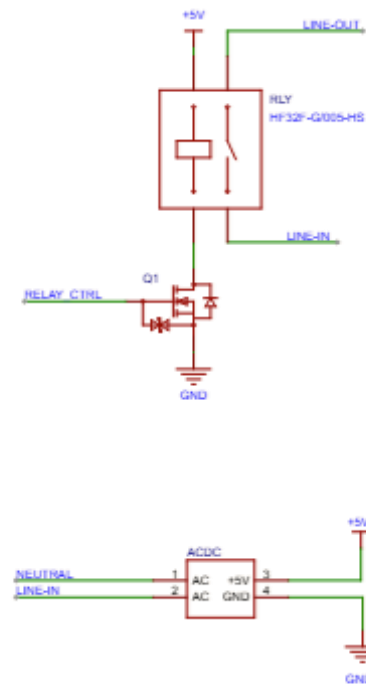
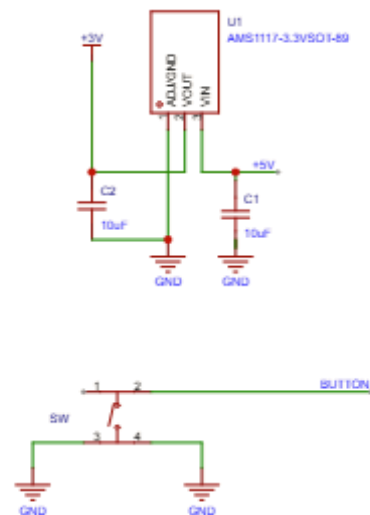
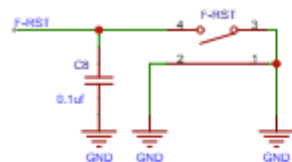
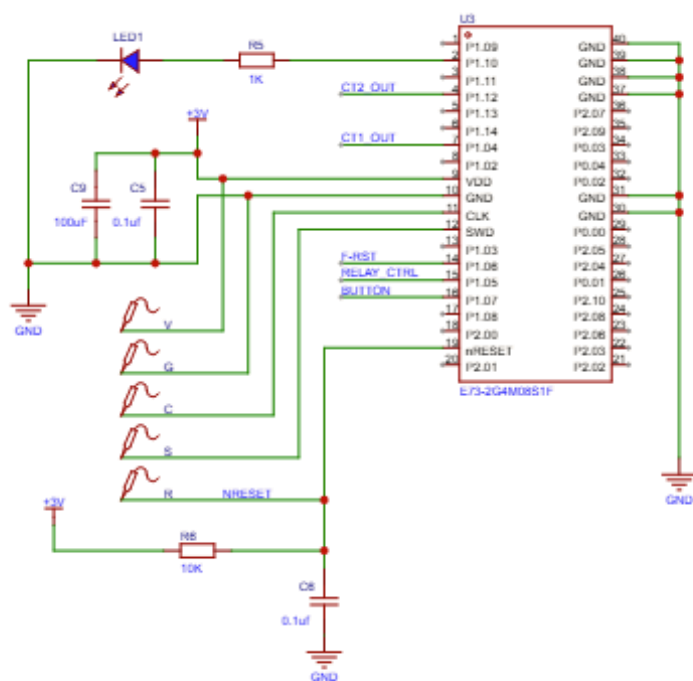
FIELD NOTE

The AINx was once wrongly set to AIN4 (really P1.11); the ADC sampled a button pull-up and read a rock-steady, fake 12-14 W.

Smart Plug

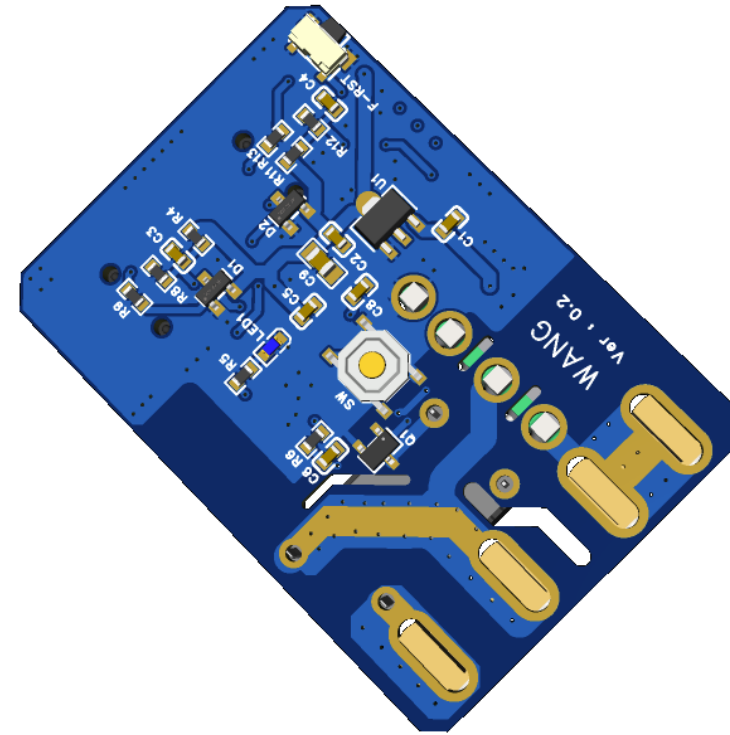
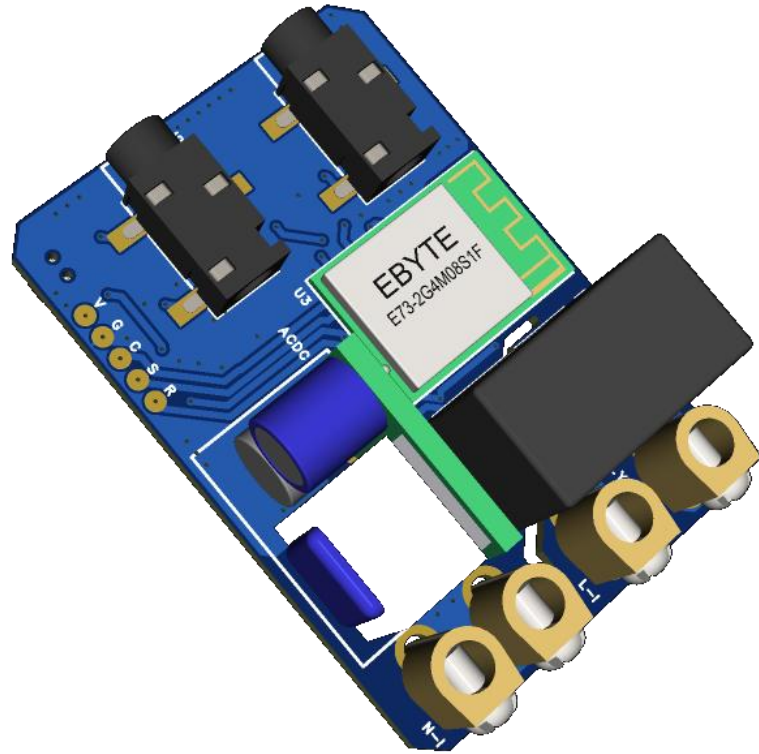
Schematic

GPIO
P1.10: LED
P1.14: Fake LED
P1.05: RELAY_CTRL
P1.04: CT1_OUT
P1.12: CT2_OUT
P1.11: OUT1
P1.07: BUTTON
P1.06: Factory Reset



Smart Plug

PCBA-3D



Three-Gang Switch

apps/3-cut-switch/

Replaces a conventional three-gang wall switch: three independent circuits, controllable from the wall or the app.

Product code	Product ID	Matter device type	Endpoints	Power / Thread role	Bootloader / OTA	Deployed
09	0x8009	0x010A On/Off Plug-in Unit	4	Mains / Thread router	MCUboot + BLE DFU	2

Hardware notes	Matter data model	Firmware modules
- Relays 1/2/3 → P1.05 / P1.07 / P1.08 - Wall buttons 1/2/3 → P1.11 / P1.12 / P1.13 (PULL_UP) - CT → P1.04, measuring total switched load current - Layout inherited from the Car_Door project	- EP1 / EP2 / EP3 one OnOff each 4 endpoints total (including EP0 root) - ElectricalPowerMeasurement (total power)	- switch_control.cpp, metering.cpp - ElectricalPowerMeasurementDelegate.cpp

GPIO map (including the board-common LED and button)					
P1.05	Relay 1 → EP1	P1.12	Wall button SW-2 → EP2	P1.10	Status LED (lit during commissioning)
P1.07	Relay 2 → EP2	P1.13	Wall button SW-3 → EP3	P1.14	Fake LED (framework placeholder, unwired)
P1.08	Relay 3 → EP3	P1.04	Current transformer CT → SAADC		
P1.11	Wall button SW-1 → EP1	P1.06	Factory-reset button (long-press >= 6 s)		

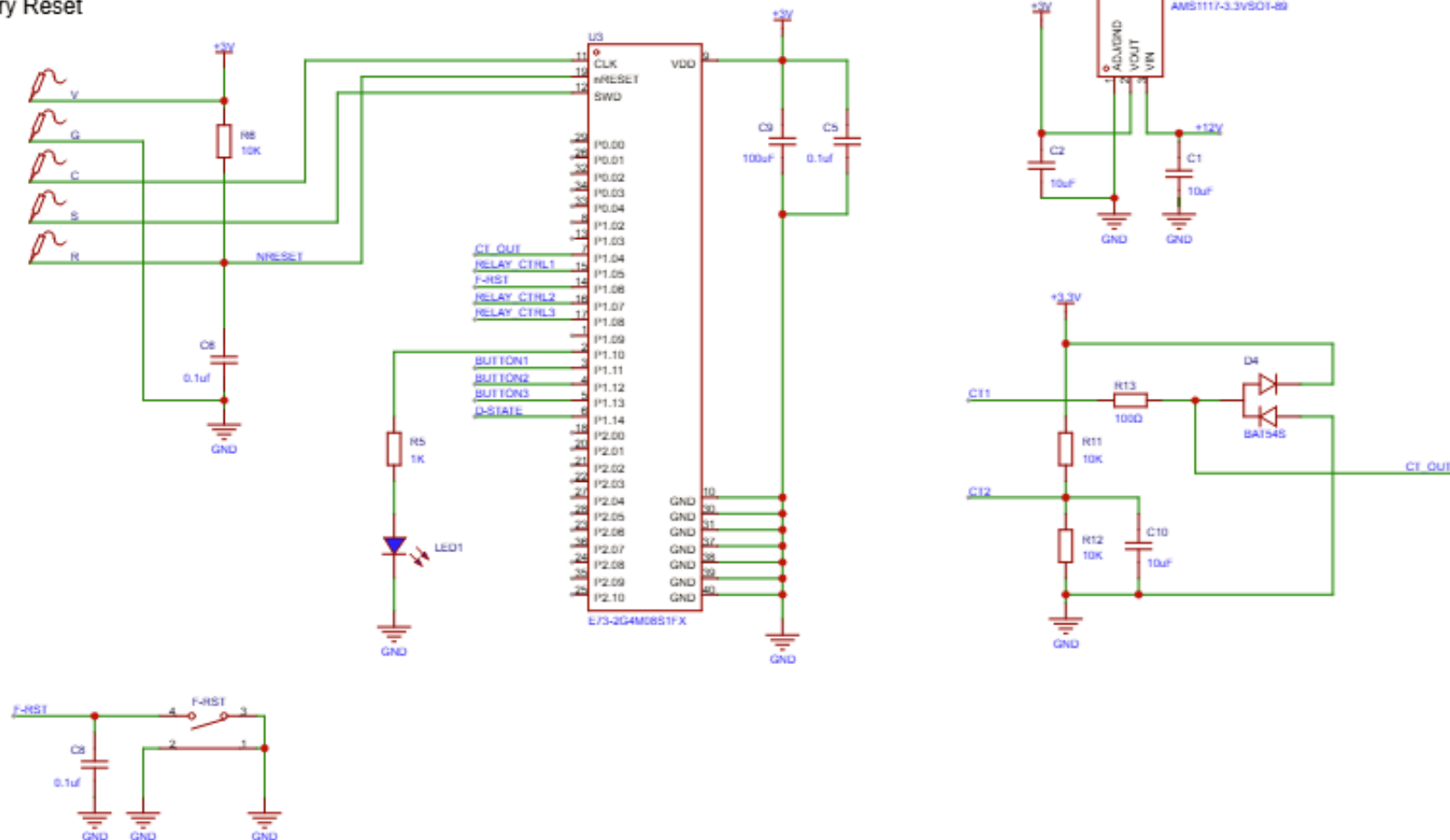
FIELD NOTE

WARNING: the overlay still declares the CT as NRF_SAADC_AIN4, but AIN4 is P1.11 (the SW-1 button). Smart_Plug and Door_Controller were corrected to AIN0; this one has not been.

Three-Gang Switch

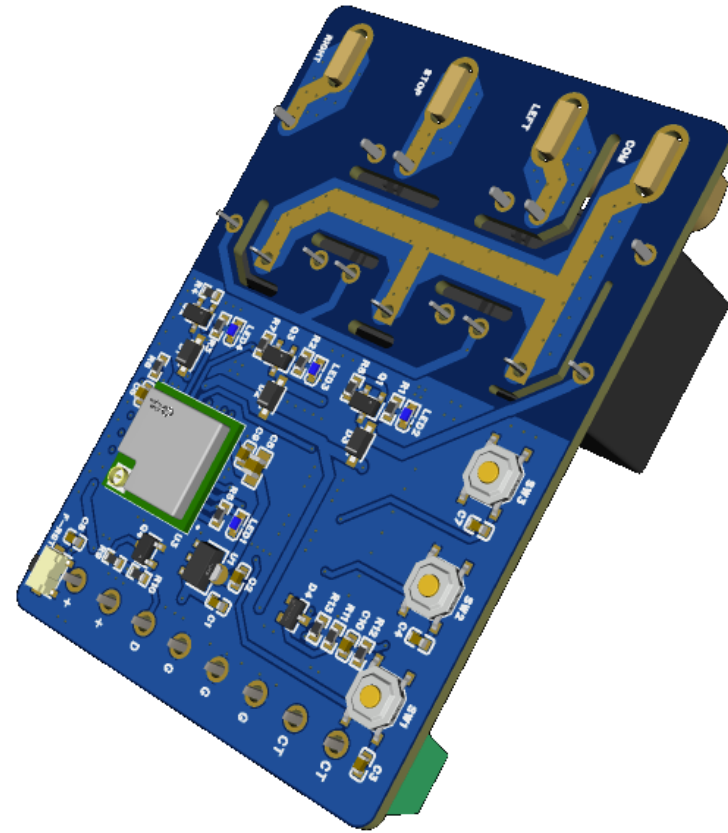
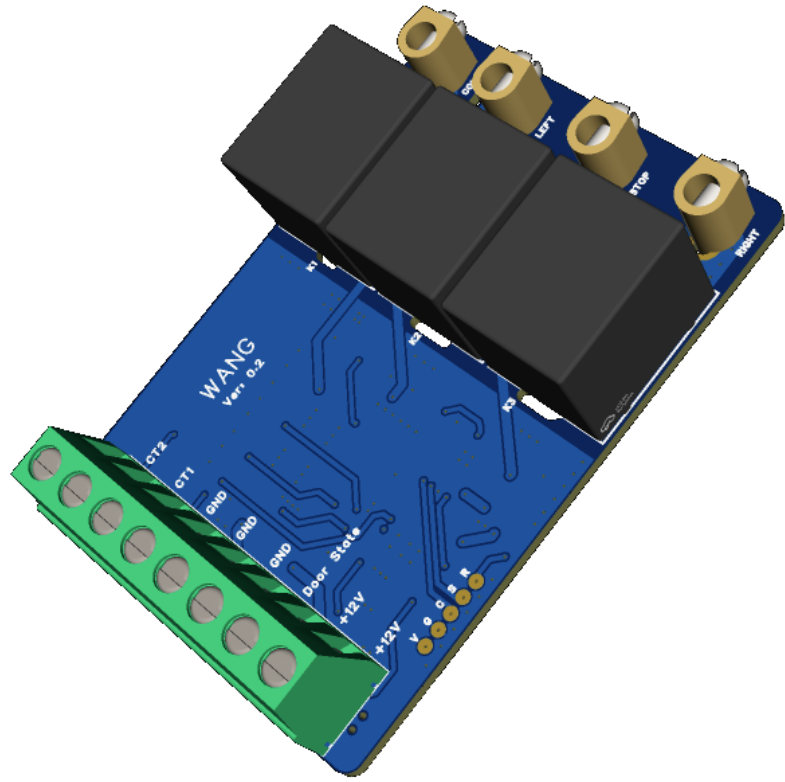
Schematic

GPIO
P1.04 : CT output; 20/1V; 1.65V bias
P1.10: LED
P1.14: Fake LED
P1.05: RELAY_CTRL1
P1.07: RELAY_CTRL2
P1.08: RELAY_CTRL3
P1.11: BUTTON1
P1.12: BUTTON2
P1.13: BUTTON3
P1.06: Factory Reset



Three-Gang Switch

PCBA-3D



Camera Power Switch

apps/camera_switch/

Simply cuts or restores power to a camera — for privacy automations like 'cameras off while I'm home'.

Product code	Product ID	Matter device type	Endpoints	Power / Thread role	Bootloader / OTA	Deployed
10	0x800A	0x010A On/Off Plug-in Unit	2	Mains / Thread router	MCUboot + BLE DFU	4

Hardware notes	Matter data model	Firmware modules
· A single relay → P0.01 · No sensors, no CT, no I2C, no ADC · Exactly one pin beyond the board-common LED and button · The simplest hardware in the whole portfolio	· EP1 OnOff · EP1 Identify · 2 endpoints total	· relay_control.cpp, ble_config_service.cpp, watchdog.cpp

GPIO map (including the board-common LED and button)			
P0.01	Relay (ACTIVE_HIGH)	P1.10	Status LED (lit during commissioning)
P1.06	Factory-reset button (long-press >= 6 s)	P1.14	Fake LED (framework placeholder, unwired)

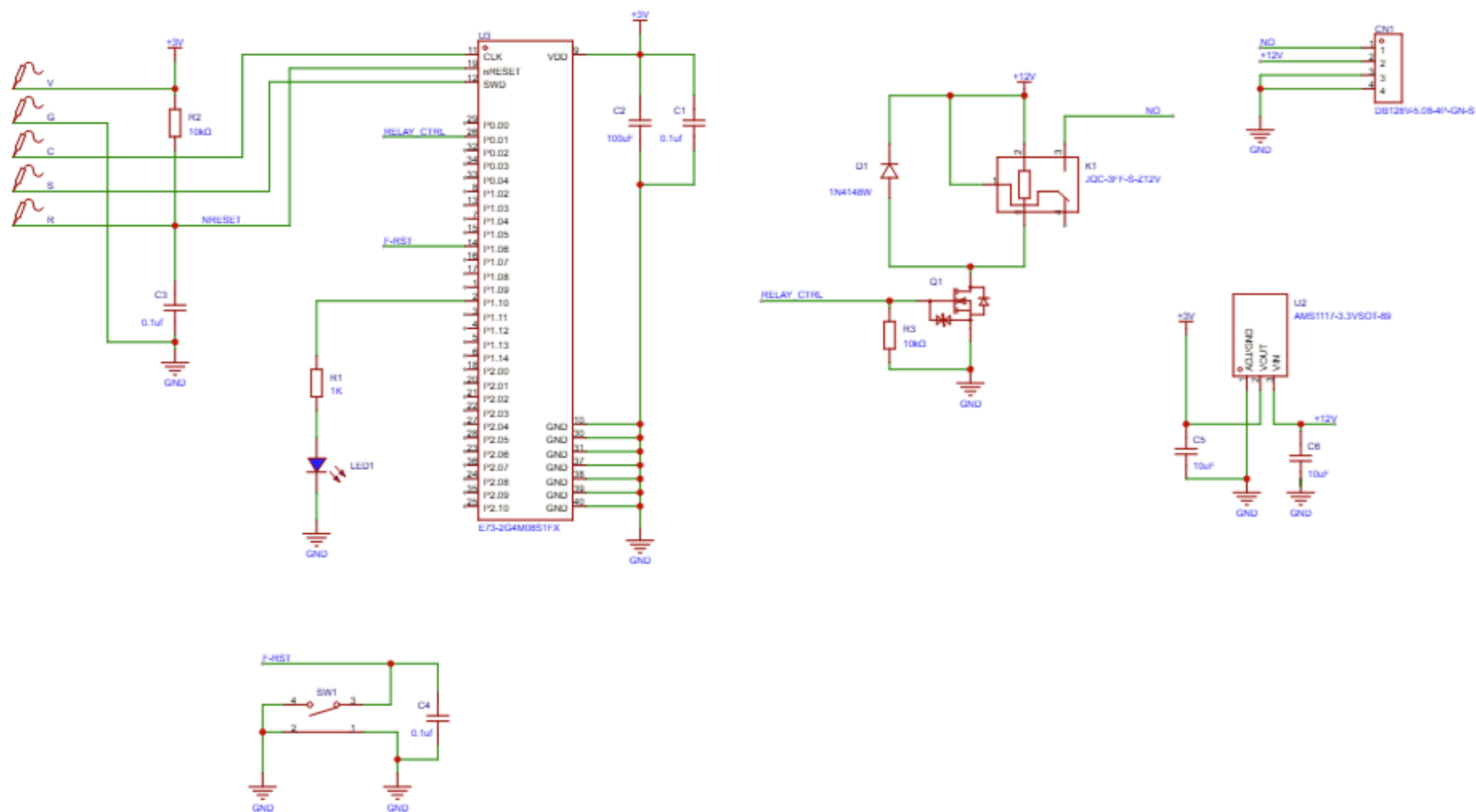
FIELD NOTE

If you want one product to copy as your very first Matter device, start from this one.

Camera Power Switch

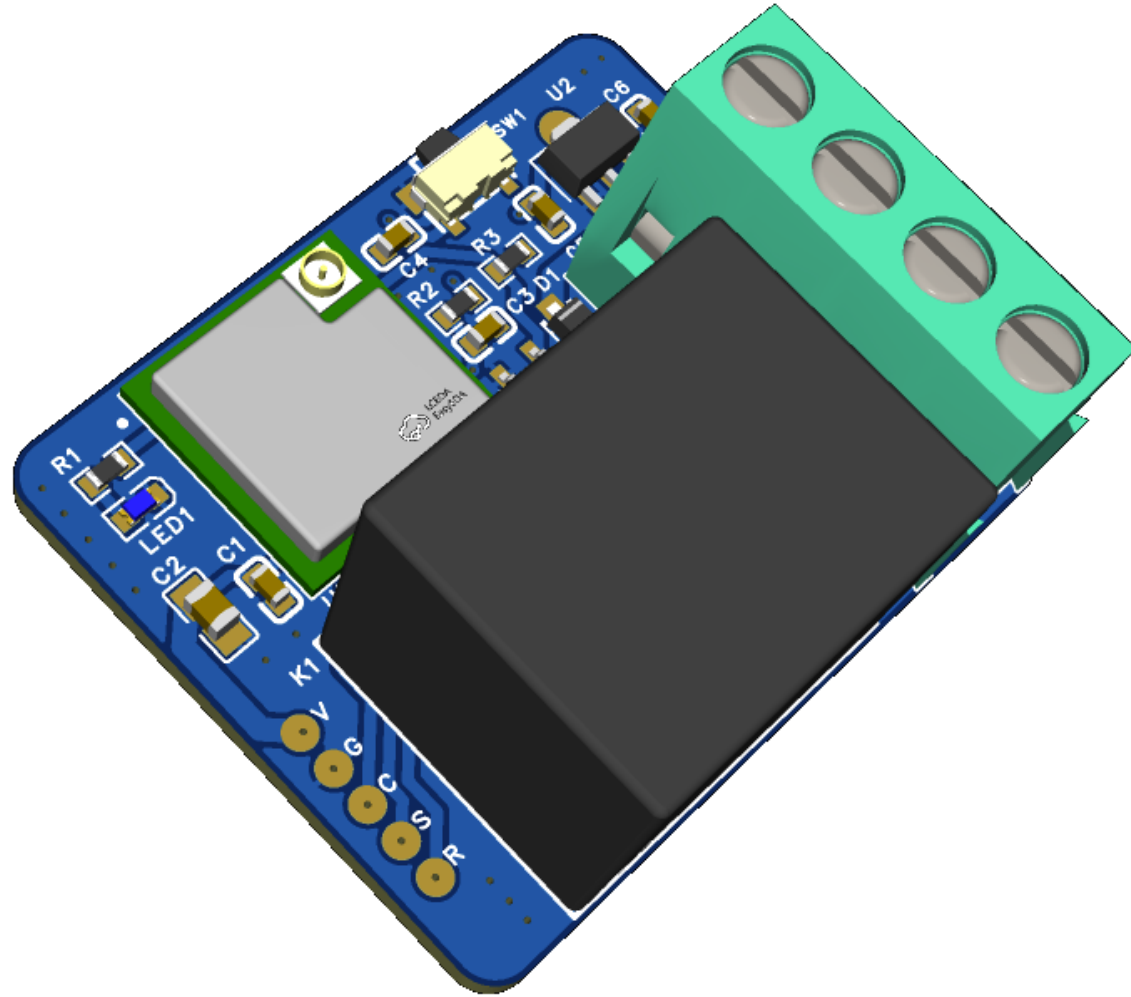
Schematic 1 / 2

GPIO
P1.10 : LED
P1.14: Fake LED to meet the requirement
of Nordic Matter Board library.
P0.01 : Relay control
P1.06 : Factory Reset



Camera Power Switch

PCBA-3D



Air Quality Sensor

apps/ENV_sensor/

Measures CO2, temperature, humidity and illuminance — the sensor with the most endpoints in the portfolio.

Product code	Product ID	Matter device type	Endpoints	Power / Thread role	Bootloader / OTA	Deployed
11	0x800B	0x002C Air Quality Sensor	5	Mains / Thread router	MCUboot + BLE DFU	2

Hardware notes	Matter data model	Firmware modules
· SCD41 @ 0x62 (CO2 + temperature + humidity) · BH1750 @ 0x23 (illuminance) · Both share i2c30: SCL P0.00 / SDA P0.01 · Mains powered — the SCD41 draws too much for a battery	· EP1 AirQuality / CarbonDioxideConcentration · EP2 TemperatureMeasurement · EP3 RelativeHumidityMeasurement · EP4 IlluminanceMeasurement	· scd41_sensor.cpp, lux_sensor.cpp · dfu_advertising.cpp

GPIO map (including the board-common LED and button)			
P0.00	I2C30 SCL (TWIM30)	—	Mains powered, no battery-sense ADC
P0.01	I2C30 SDA (TWIM30)	P1.06	Factory-reset button (long-press >= 6 s)
I2C 0x62	SCD41: CO2 + temperature + humidity	P1.10	Status LED (lit during commissioning)
I2C 0x23	BH1750 lux meter	P1.14	Fake LED (framework placeholder, unwired)

FIELD NOTE

Once shared a hard-coded serial (DZNRF54L00105) with the gesture sensor and got fused into one device in HA.

Air Quality Sensor

Schematic

GPIO

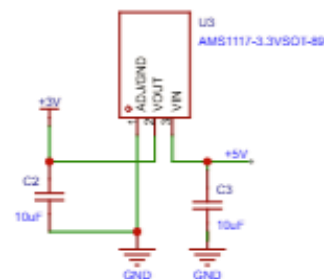
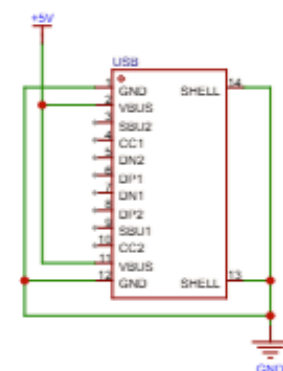
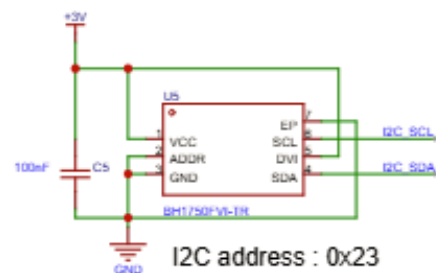
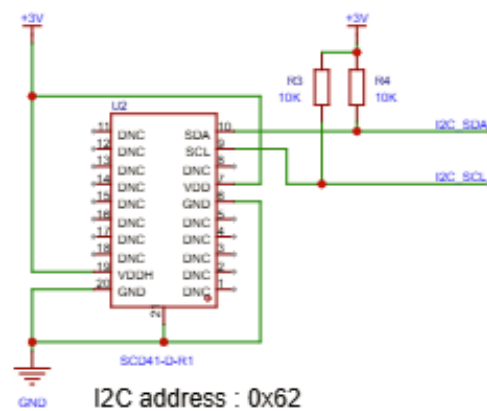
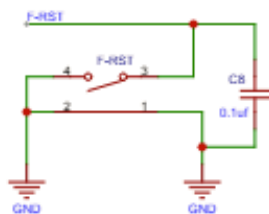
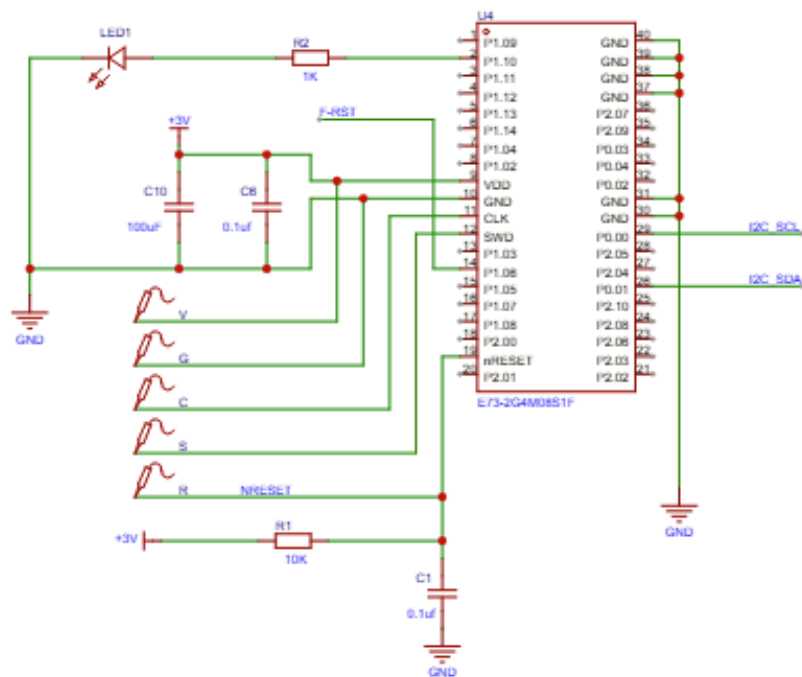
P1.10 : LED

P1.14 : Fake LED to meet Nordic Matter Board library.

P0.00 : I2C_SCL

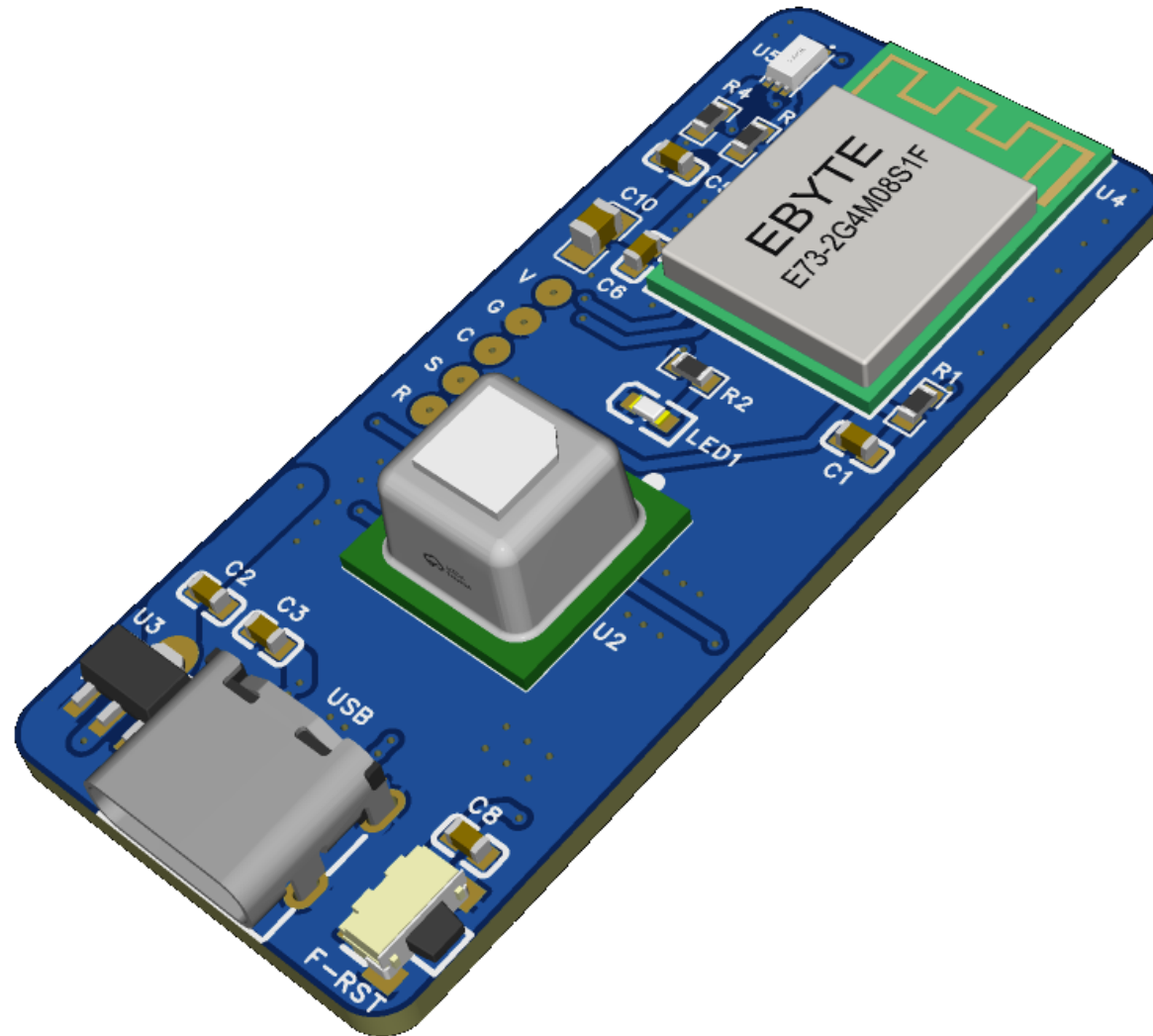
P0.01 : I2C_SDA

P1.06 : Factory Reset



Air Quality Sensor

PCBA-3D



Pulse Door Lock

apps/Pulse_Lock/

Sends a short pulse to an electric strike to release the door, presented as a standard Matter door lock.

Product code	Product ID	Matter device type	Endpoints	Power / Thread role	Bootloader / OTA	Deployed
12	0x800C	0x000A Door Lock	2	Mains / Thread router	MCUboot + BLE DFU	0

Hardware notes	Matter data model	Firmware modules
· Pulse output → P0.01 · No sensor — currently open-loop · Does not report physical lock state · Same pinout as camera_switch	· EP1 DoorLock (LockState, Lock/Unlock) · 2 endpoints total	· pulse_switch.cpp (pulse timing) · reboot_fix.cpp, watchdog.cpp

GPIO map (including the board-common LED and button)			
P0.01	Pulse output → strike (ACTIVE_HIGH)	P1.10	Status LED (lit during commissioning)
P1.06	Factory-reset button (long-press >= 6 s)	P1.14	Fake LED (framework placeholder, unwired)

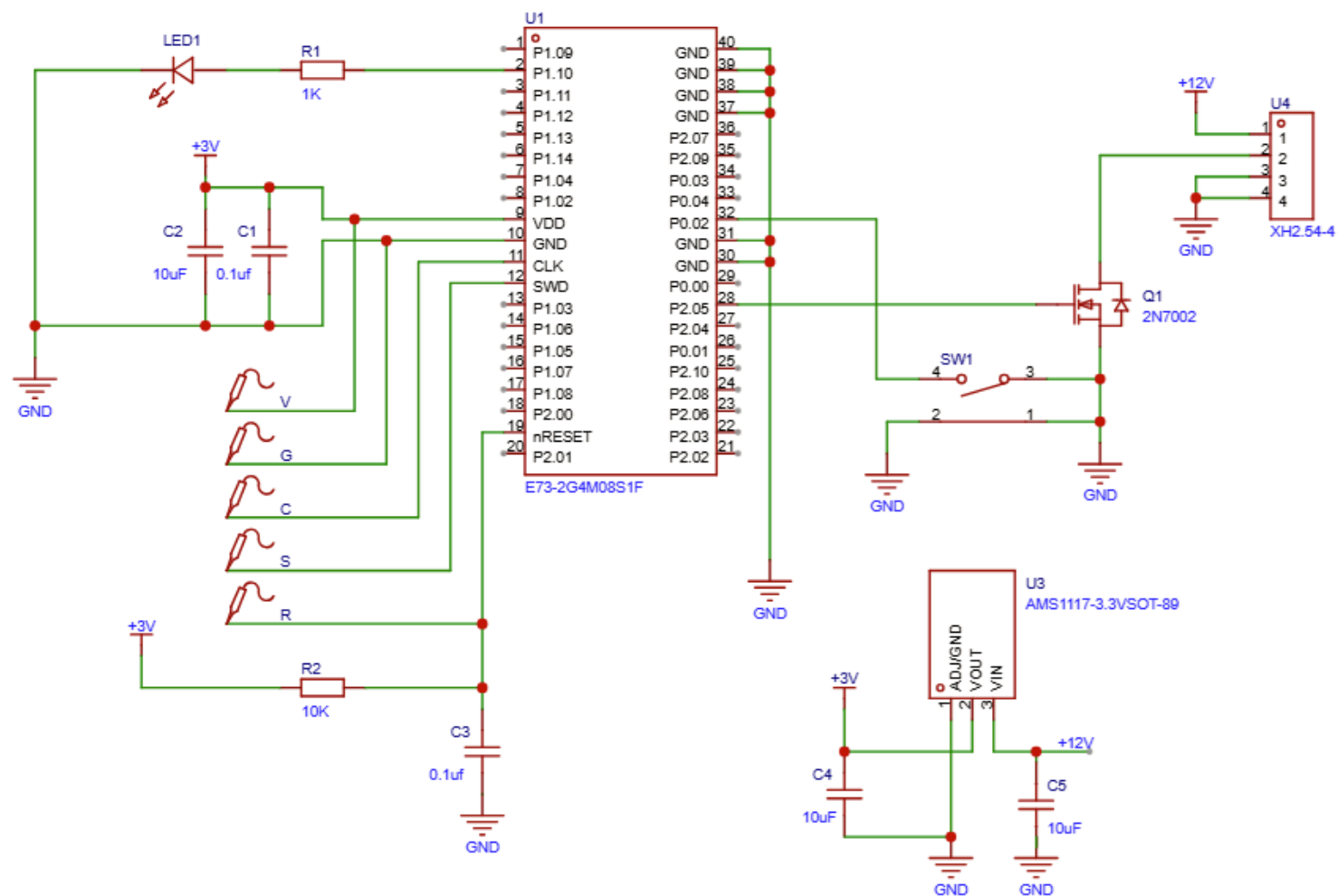
FIELD NOTE

Firmware is complete and builds, but has not been installed on any unit yet (0 deployed).

Pulse Door Lock

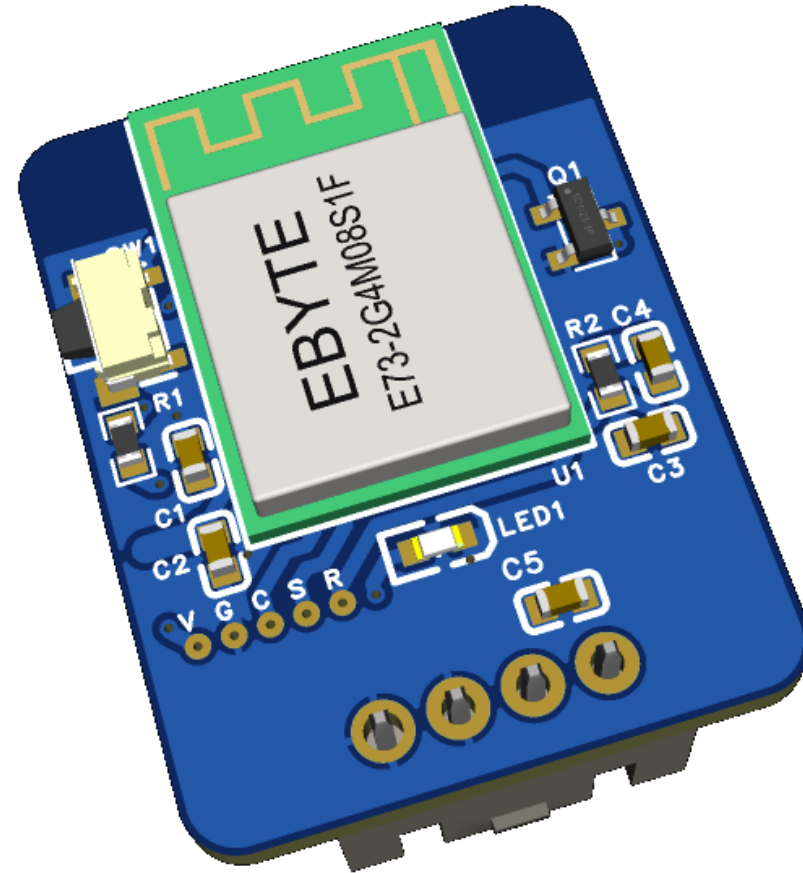
Schematic

GPIO
P1.10 : LED
P2.05 : HALL Sensor
P0.02 : Factory Reset



Pulse Door Lock

PCBA-3D



Fan Controller

apps/fan_control/

Controls an existing fan by emulating presses on its own buttons, then reads feedback taps to infer the real state.

Product code	Product ID	Matter device type	Endpoints	Power / Thread role	Bootloader / OTA	Deployed
15	0x800F	0x002B Fan	2	Mains / Thread router	MCUboot + BLE DFU	2

Hardware notes	Matter data model	Firmware modules
· All three emulated buttons sit on port 2 · Both feedback taps sit on port 1 (interrupt-capable) · The spin PWM tap moved P1.03 → P1.05 for a hardware check · It never drives the motor — it only presses buttons	· EP1 FanControl (speed, mode) · EP1 OnOff · Swing exposed via RockSetting / custom attributes	· fan_control.cpp (emit emulated presses) · fan_monitor.cpp (measure PWM period to derive speed) · dfu_advertising.cpp

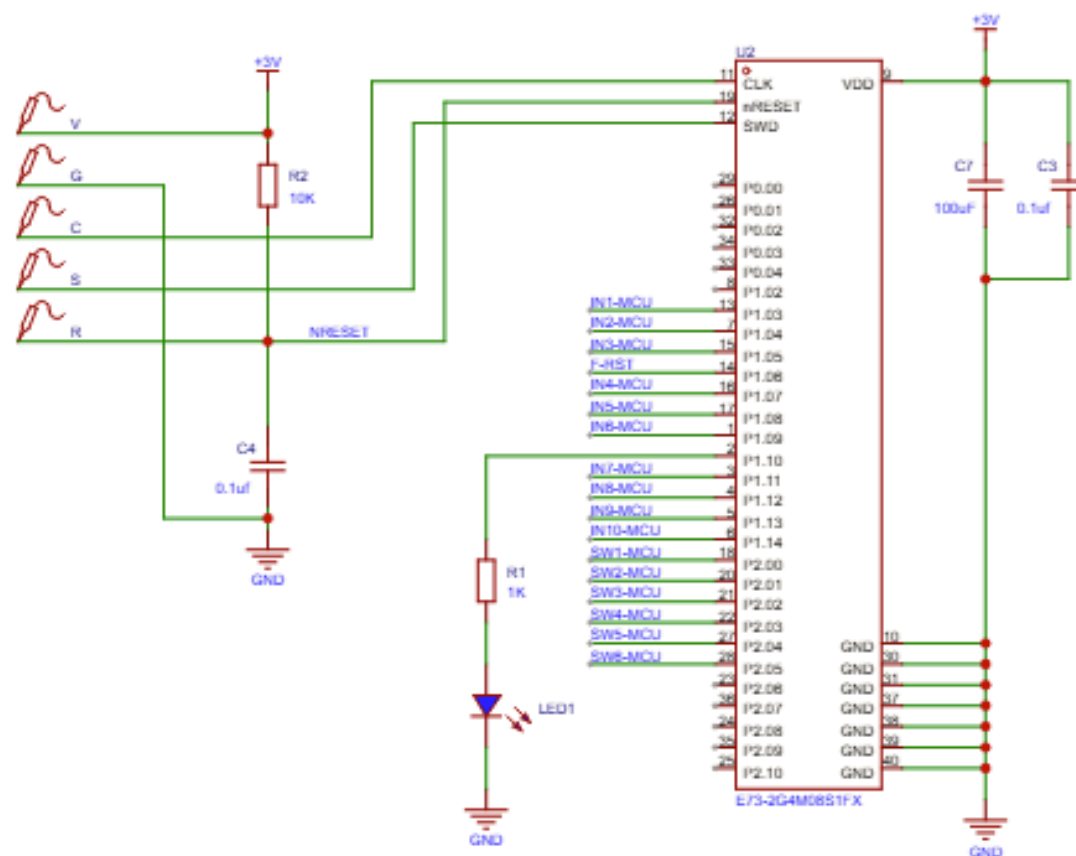
GPIO map (including the board-common LED and button)					
P2.00	Emulated button: fan on/off	P1.05	Feedback: spin PWM (period → speed level)	P1.06	Factory-reset button (long-press >= 6 s)
P2.01	Emulated button: swing on/off	P1.04	Feedback: swing pulse (edge presence)	P1.10	Status LED (lit during commissioning)
P2.02	Emulated button: speed step +1	回授腳	Both feedback pins GPIO_PULL_UP	P1.14	Fake LED (framework placeholder, unwired)

FIELD NOTE

Because it presses buttons rather than driving the motor, state MUST be synced from feedback or the app will drift out of step.

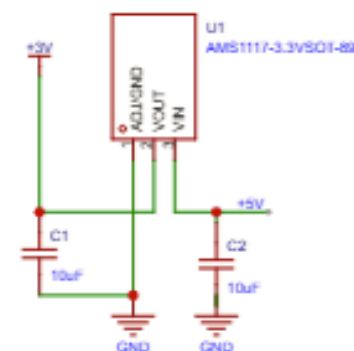
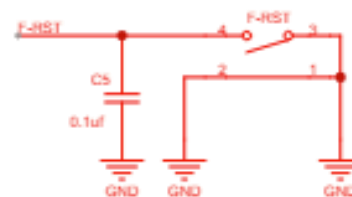
Fan Controller

Schematic



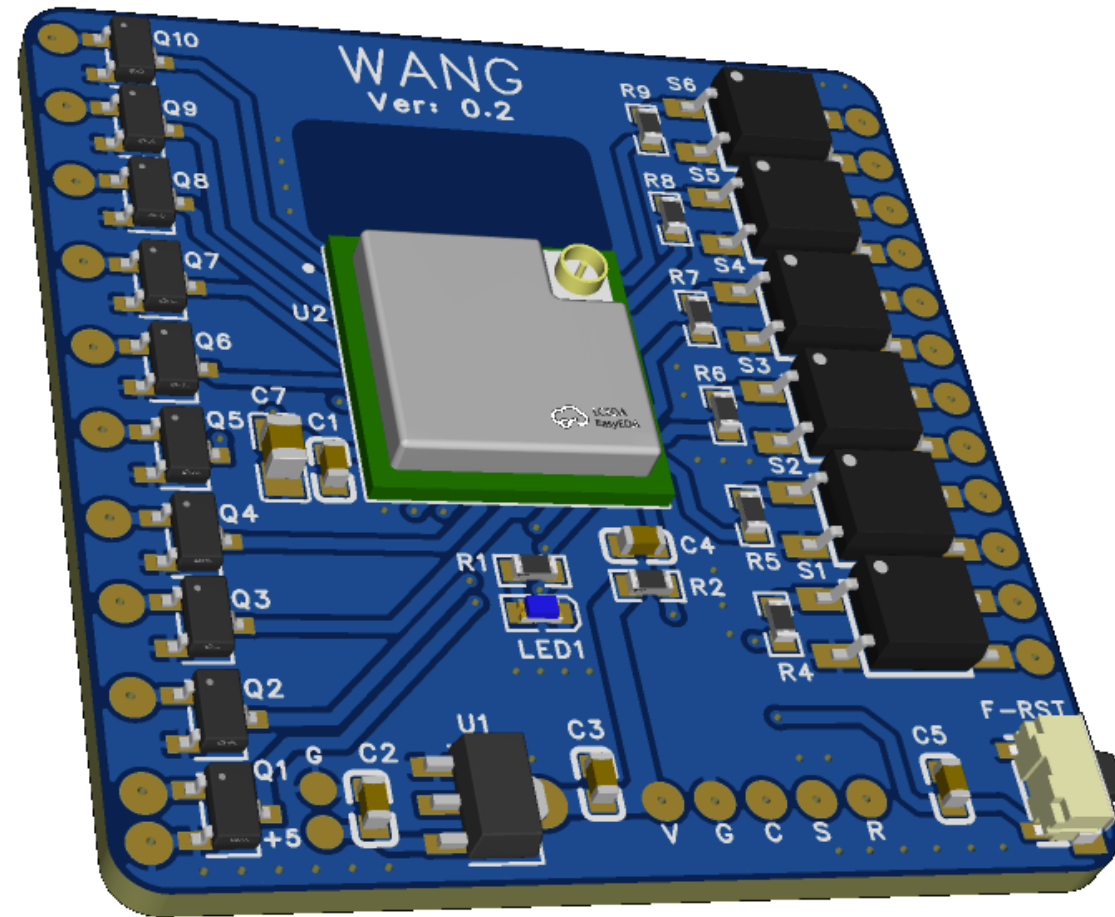
GPIO
P1.10: LED
P1.14: Fake LED
P1.03: IN1-MCU
P1.04: IN2-MCU
P2.00: BUTTON1
P2.01: BUTTON2
P2.02: BUTTON3
P1.06: Factory Reset

to detect spin level
to detect swing on/off
for FAN on/off
for swing on/off
for spin level control, one level pre trigger



Fan Controller

PCBA-3D



Squeezing 12 fan speeds into 4

Home Assistant and HomeKit use a percentage, but the Bemfa bridge to XiaoAi carries only speeds 1-4:

```
Bemfa fan topic (type suffix 003) message format:  
  on|off # speed (1-4) # swing
```

- Our fans expose 12 speeds (percentage_step = 8.333%), so 12 must collapse to 4
- Speed 0 is NOT valid in the protocol — a voice assistant reads on#0#... as OFF
- So while the fan is on it never reports 0; the floor is gear 1

Gear (XiaoAi)	1	2	3	4
Percentage (HA / HK)	25%	50%	75%	100%

Two conversion paths, chosen per device

custom_components/bemfa/sync_fan.py picks a path from what the entity can do:

Path A - percentage

```
report: clamp(ceil(pct / 25), 1..4)
command: fan.set_percentage(gear * 25)
```

Path B - preset mode

```
low:1    medium:2
high:3   auto:4
```

- Why two? Two fans of the SAME model and firmware behave differently:
 - one reports percentage = 0 permanently, even while running on a preset → path B
 - the other reports real percentages → path A
- WARNING: gear 4 maps to 'auto', not to 'fastest' — high actually sits at gear 3
 - to make maximum mean high, change one line in PRESET_TO_GEAR

A HACS update SILENTLY overwrites sync_fan.py. If gear behaviour regresses, check that file before anything else — and a full HA restart is required to load changes.

Garage Door Controller

apps/Door_Controller/

Drives a roller-door motor open/stop/close, reports the door's real position, and also receives the original 433 MHz remote.

Product code	Product ID	Matter device type	Endpoints	Power / Thread role	Bootloader / OTA	Deployed
17	0x8011	0x0202 Window Covering	3	Mains / Thread router	MCUboot + BLE DFU	1

Hardware notes	Matter data model	Firmware modules
- Dual AS5600 vernier encoders: FINE 1:1, COARSE 1:31 - FINE on hardware TWIM30 (the only serial on port 0) - COARSE on bit-banged gpio-i2c using the NFC pins P1.02/03 - CC1101 433 MHz OOK receive, reads RSSI / LQI - CT → P1.04 = AIN0 (confirmed 2026-08-26, not AIN4) - Buttons P1.11–13 and reed P1.14 consume AIN4–AIN7	- EP1 WindowCovering (open/stop/close, position) - EP2 BooleanState (reed contact) - ElectricalPowerMeasurement (motor current)	- door_control.cpp (motion state machine) - rf_receiver.cpp (CC1101 OOK decode) - encoder.cpp, contact_sensor.cpp, metering.cpp

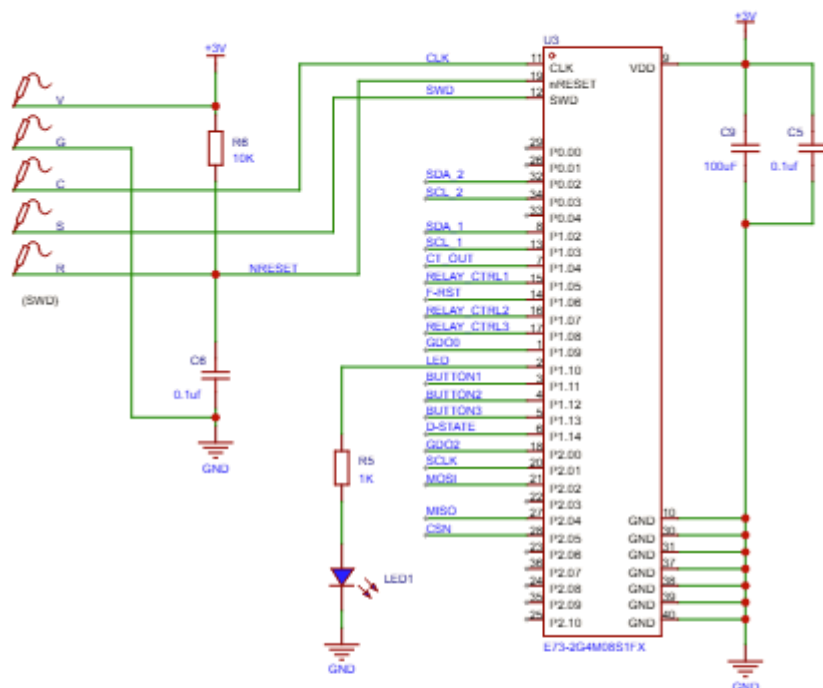
GPIO map (including the board-common LED and button)							
P1.05	Relay: close	P1.13	Button: open	P2.02	CC1101 MOSI	P1.03	gpio-i2c SCL → AS5600 COARSE
P1.07	Relay: stop	P1.14	Reed contact (through a 2N7002 N-FET)	P2.04	CC1101 MISO (= GDO1)	P1.04	Current transformer CT → SAADC AIN0
P1.08	Relay: open	P1.09	CC1101 GDO0: OOK data	P0.02	I2C30 SDA → AS5600 FINE encoder	P1.06	Factory-reset button (long-press >= 5s)
P1.11	Button: close	P2.05	CC1101 CSN	P0.03	I2C30 SCL → AS5600 FINE encoder	P1.10	Status LED (lit during commissioning)
P1.12	Button: stop	P2.01	CC1101 SCK	P1.02	gpio-i2c SDA → AS5600 COARSE encoder	P0.04	Fake LED — remapped because P1.14 is taken

FIELD NOTE

The most complex product here: relays, RF receive, both hardware and bit-banged I2C, CT metering and a reed input — it uses nearly every pin.

Garage Door Controller

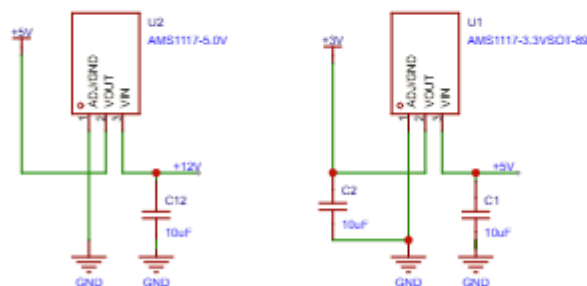
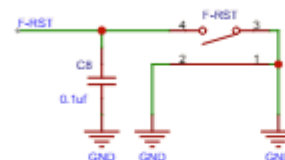
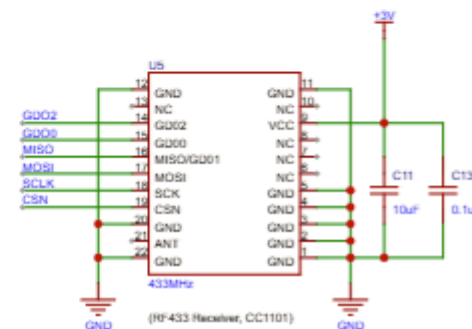
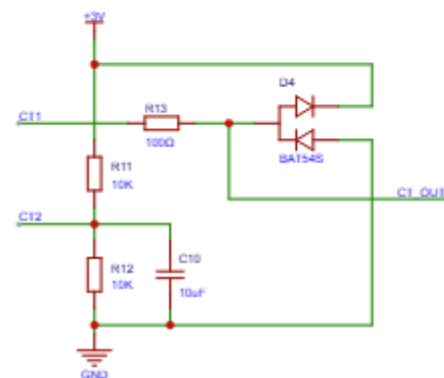
Schematic 1 / 2



GPIO

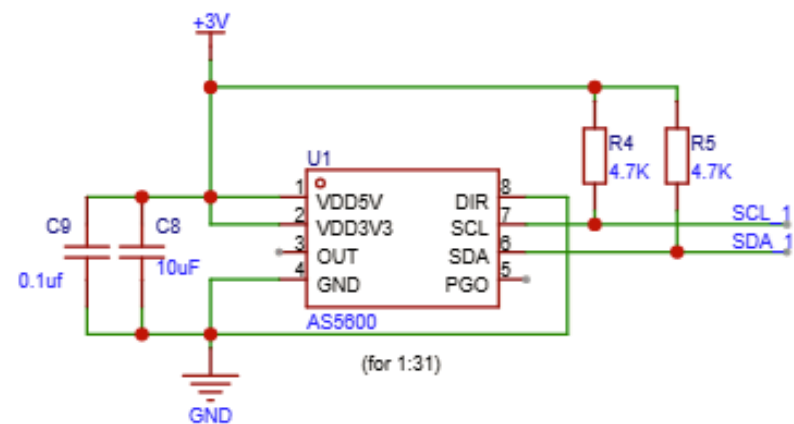
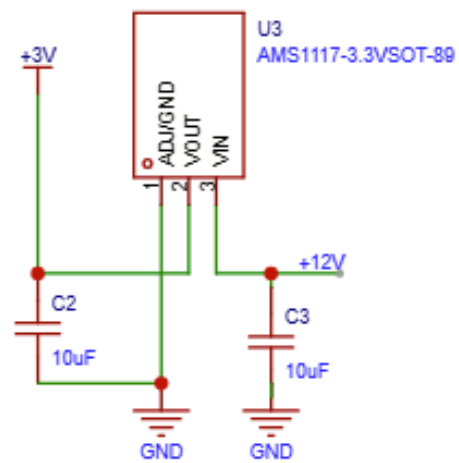
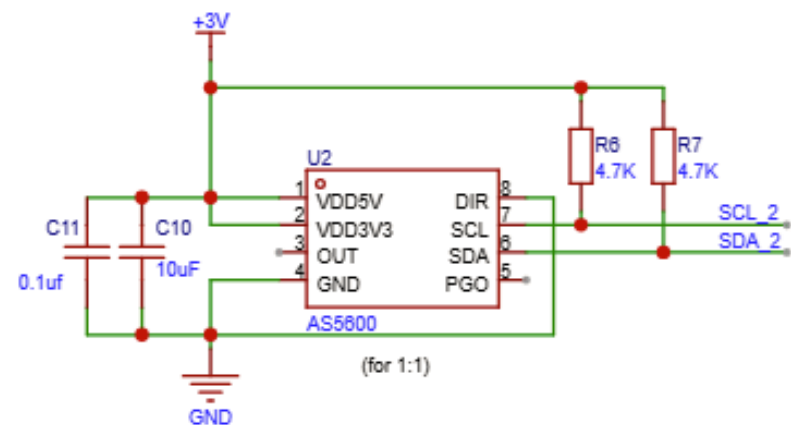
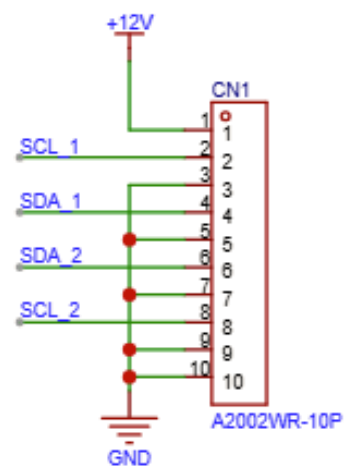
P1.04 : CT output; 20A/1V; 1.65V bias
 P1.10: LED
 P1.14: D-STATE (Door sensor)
 P1.05: RELAY_CTRL1 (Close)
 P1.07: RELAY_CTRL2 (Stop)
 P1.08: RELAY_CTRL3 (Open)
 P1.11: BUTTON1 (Close)
 P1.12: BUTTON2 (Stop)
 P1.13: BUTTON3 (Open)
 P1.06: Factory Reset

P0.02 : SDA_2 (I2C for main AS5600)
 P0.03 : SCL_2 (I2C for main AS5600)
 P1.02 : SDA_1 (I2C for 2nd AS5600)
 P1.03 : SCL_1 (I2C for 2nd AS5600)
 P2.00 : GDO2 (for CC1101)
 P2.01 : SCLK (for CC1101)
 P2.02 : MOSI (for CC1101)
 P2.04 : MISO (for CC1101)
 P2.05 : CSN (for CC1101)
 P1.09 : GDO0 (for CC1101)



Garage Door Controller

Schematic 2/ 2



Temp / Humidity Router

apps/temphumidity-router/

Senses the same things as the battery version, but mains-powered — so it also acts as a Thread router and extends mesh coverage.

Product code	Product ID	Matter device type	Endpoints	Power / Thread role	Bootloader / OTA	Deployed
18	0x8012	0x0302 Temperature Sensor	3	Mains / Thread router	MCUboot + BLE DFU	1

Hardware notes	Matter data model	Firmware modules
· SHT35 @ 0x44 (temperature + humidity) · i2c30: SCL P0.00 / SDA P0.01 · Mains powered, no battery-monitoring circuit · Nearly identical hardware to the battery version	· EP1 TemperatureMeasurement · EP2 RelativeHumidityMeasurement · No ICD — always on, forwards packets for others	· temp_humidity_sensor.cpp, ble_config_service.cpp, watchdog.cpp

GPIO map (including the board-common LED and button)			
P0.00	I2C30 SCL (TWIM30)	P1.06	Factory-reset button (long-press >= 6 s)
P0.01	I2C30 SDA (TWIM30)	P1.10	Status LED (lit during commissioning)
I2C 0x44	SHT35 temperature + humidity	P1.14	Fake LED (framework placeholder, unwired)
—	Mains powered, no battery-sense ADC		

FIELD NOTE

A genuinely useful trick: put a mains version of a sensor at the edge of the mesh and get coverage for free.

Schematic

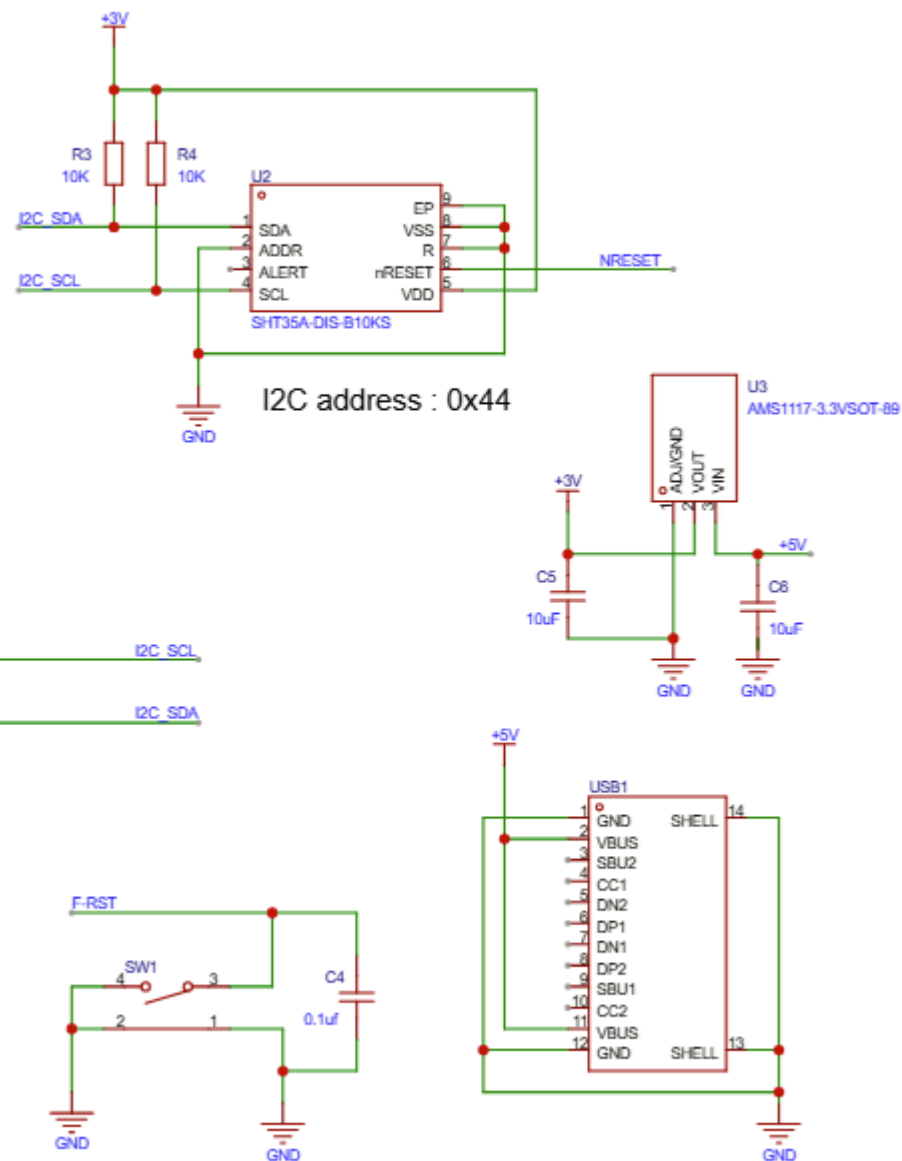
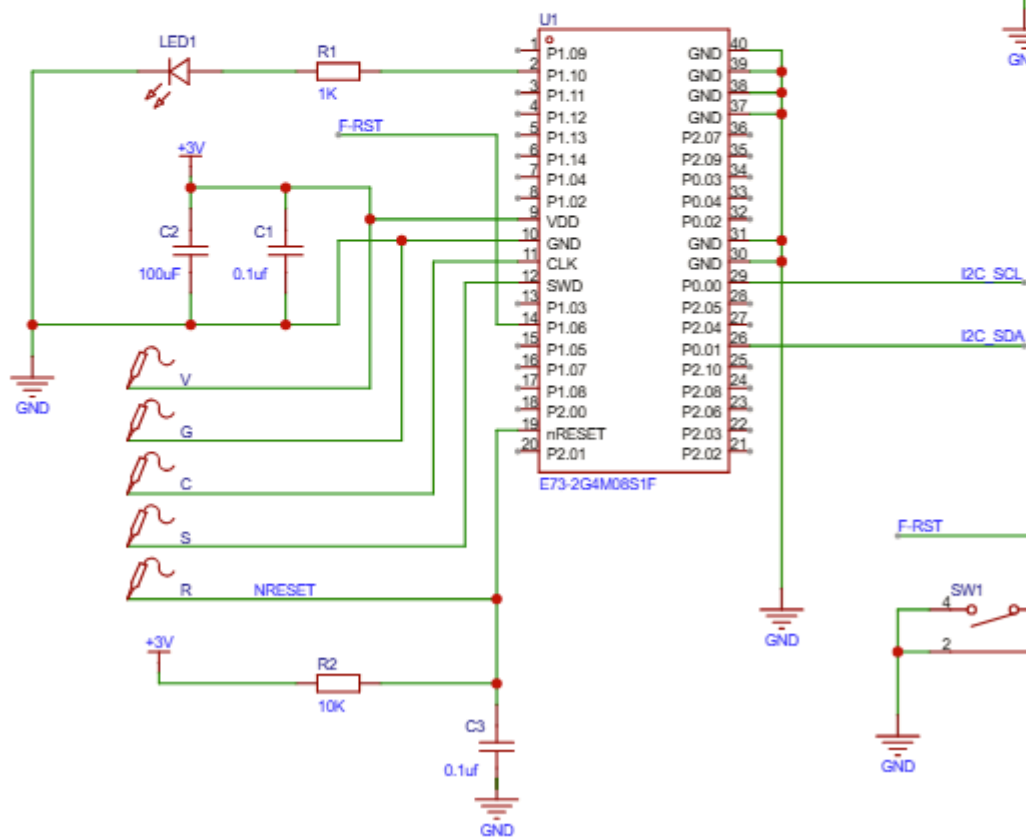
P1.10 : LED

P1.14 : Fake

P0.00 : I2C SCL

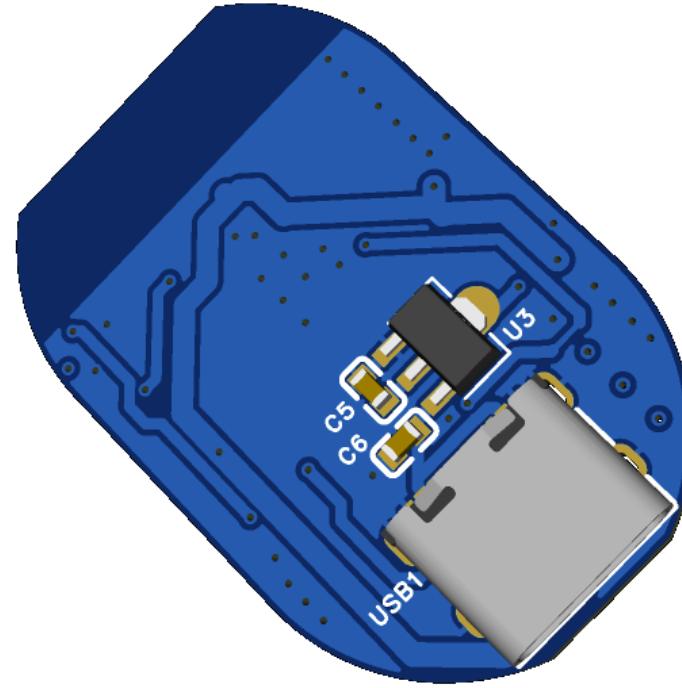
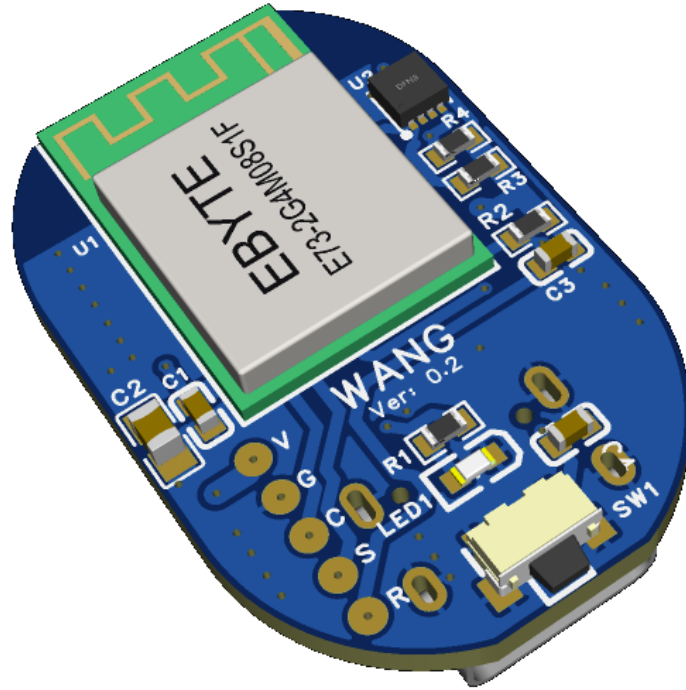
P0.01 : I2C SDA

P1.06 : Factory Reset



Temp / Humidity Router

PCBA-3D



Temp / Humidity Sensor

apps/TempHumidity_Sensor_SHT35/

Split out from the BMP280 version: same SHT35, with the pressure sensor and its endpoint removed.

Product code	Product ID	Matter device type	Endpoints	Power / Thread role	Bootloader / OTA	Deployed
19	0x8013	0x0302 Temperature Sensor	3	Battery / ICD sleepy	none (v1, no bootloader)	1

Hardware notes	Matter data model	Firmware modules
· SHT35 @ 0x44 (temperature + humidity) · i2c30: SCL P0.00 / SDA P0.01 · SAADC internal VDD for battery · Only difference from the BMP280 version: no part at 0x76	· EP1 TemperatureMeasurement · EP2 RelativeHumidityMeasurement · EP1 PowerSource (battery)	· temp_humidity_sensor.cpp, battery_monitor.cpp, watchdog.cpp

CR2032 life estimate	Usable ~200 mAh	Wake period 300 s	Avg current ~5.3 µA	Theoretical ~4.3 yr	Realistic 1.5–2 yr	Dominant load SHT35 read every 300 s, ~15 ms
-------------------------	--------------------	----------------------	------------------------	------------------------	-----------------------	---

GPIO map (including the board-common LED and button)

P0.00	I2C30 SCL (TWIM30)	P1.06	Factory-reset button (long-press >= 6 s)
P0.01	I2C30 SDA (TWIM30)	P1.10	Status LED (lit during commissioning)
I2C 0x44	SHT35 temperature + humidity	P1.14	Fake LED (framework placeholder, unwired)
SAADC	Battery voltage (internal VDD, no external divider)		

FIELD NOTE

Splitting this product hit Lesson 4: its new PID fell outside the copied, stale CD and it could not be commissioned at all.

Schematic 1 / 2

P1.10 : LED

P1.14 : Fake LED to meet Nordic Matter Board library.

P0.00 : I2C SCL

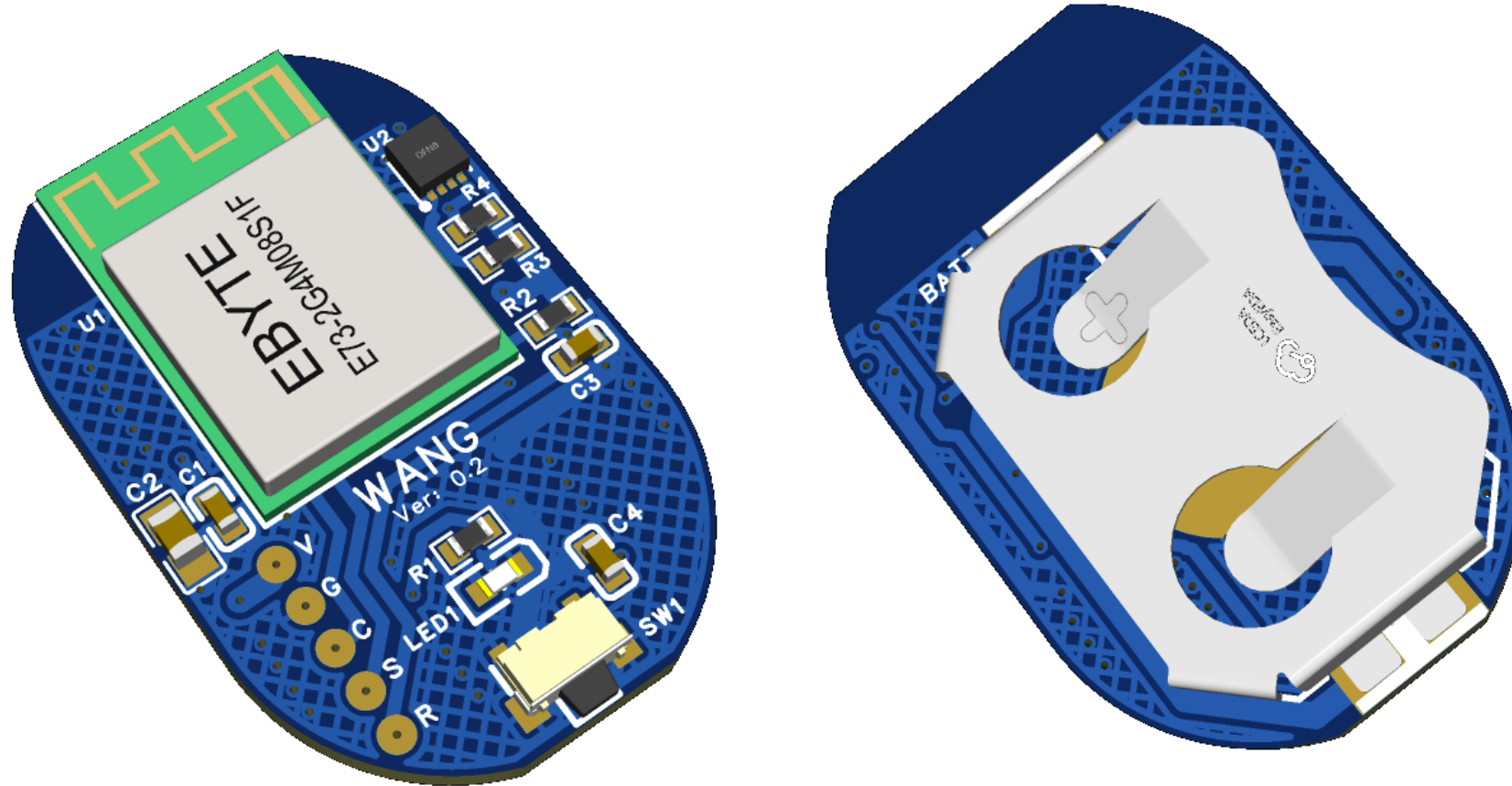
P0.01 : I2C_SDA

P1.06 : Factory Reset



Temp / Humidity Sensor

Schematic 2 / 2



Smart Plug (HLW8032 metering)

apps/AC_SmartPlug/

Functionally an ordinary smart plug. The one difference from Smart_Plug is how it meters: this version uses an HLW8032 meter IC and reads BOTH voltage and current, where Smart_Plug uses a CT and reads current only.

Product code	Product ID	Matter device type	Endpoints	Power / Thread role	Bootloader / OTA	Deployed
20	0x8014	0x010A On/Off Plug-in Unit	2	Mains / Thread router	MCUboot + BLE DFU	1

Hardware notes	Matter data model	Firmware modules
· Metered by an HLW8032 meter IC: voltage AND current · This product has NO ADC at all — the &adc node is deliberately absent · The HLW8032 free-runs, streaming a 24-byte frame every ~50 ms · WARNING: P1.04 / P1.05 swap roles versus Smart_Plug · Both buttons are app-owned and share one handler	· EP1 OnOff (state persisted, restored across reboot) · EP1 ElectricalPowerMeasurement (power / current) · EP1 Identify	· relay_control.cpp, metering.cpp · ElectricalPowerMeasurementDelegate.cpp · ble_config_service.cpp, watchdog.cpp

GPIO map (including the board-common LED and button)					
P1.04	Latching relay (ACTIVE_HIGH)	P1.07	User button (PULL_UP / ACTIVE_LOW)	UART	4800 baud, 8 data bits, EVEN parity, 1 stop bit
P1.05	State LED: mirrors the relay	P1.06	On-module dev button (sw0 pin, app-owned)	P1.10	Status LED (lit during commissioning)
P1.08	Power LED: driven HIGH once at init	P1.11	UART20 RX ← HLW8032 (through an EL357 opto)	P1.14	Fake LED (framework placeholder, unwired)

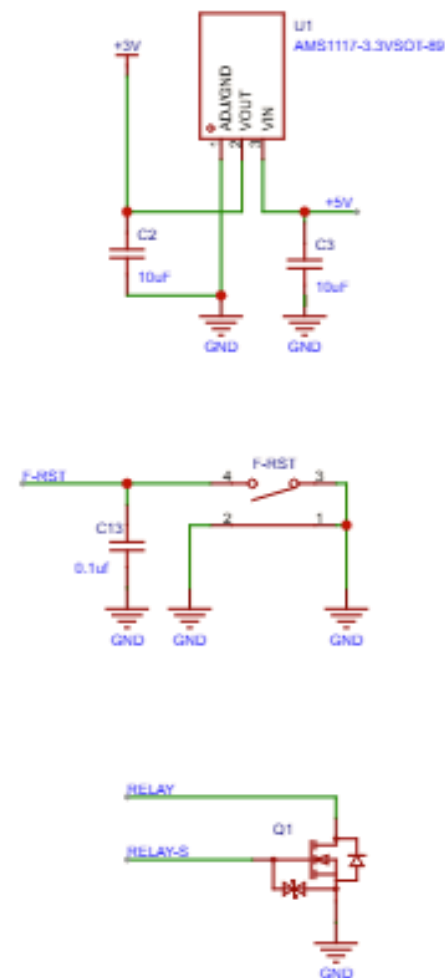
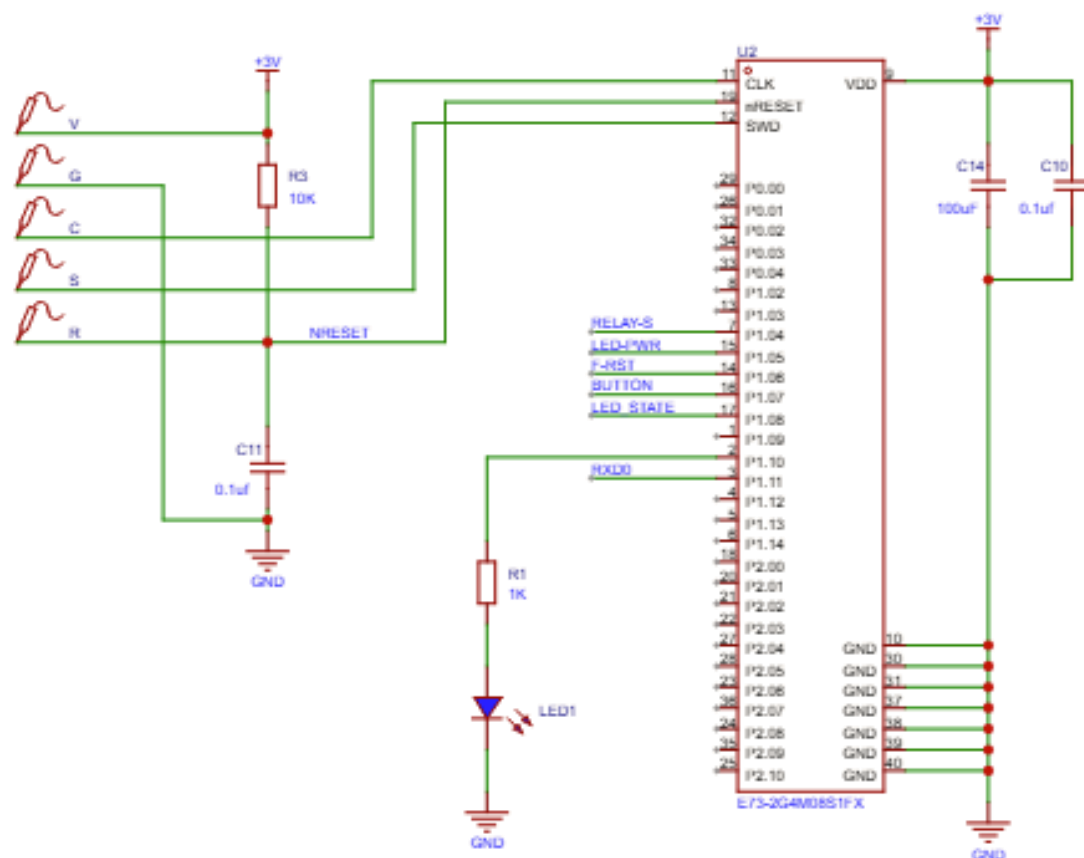
FIELD NOTE

On 2026-09-06 the power and state LEDs were swapped (P1.05 <-> P1.08) for a layout fix. The DT properties are named by FUNCTION, not by pin, so no C++ changed — but boards built before that date are wired the other way round.

Smart Plug (HLW8032 metering)

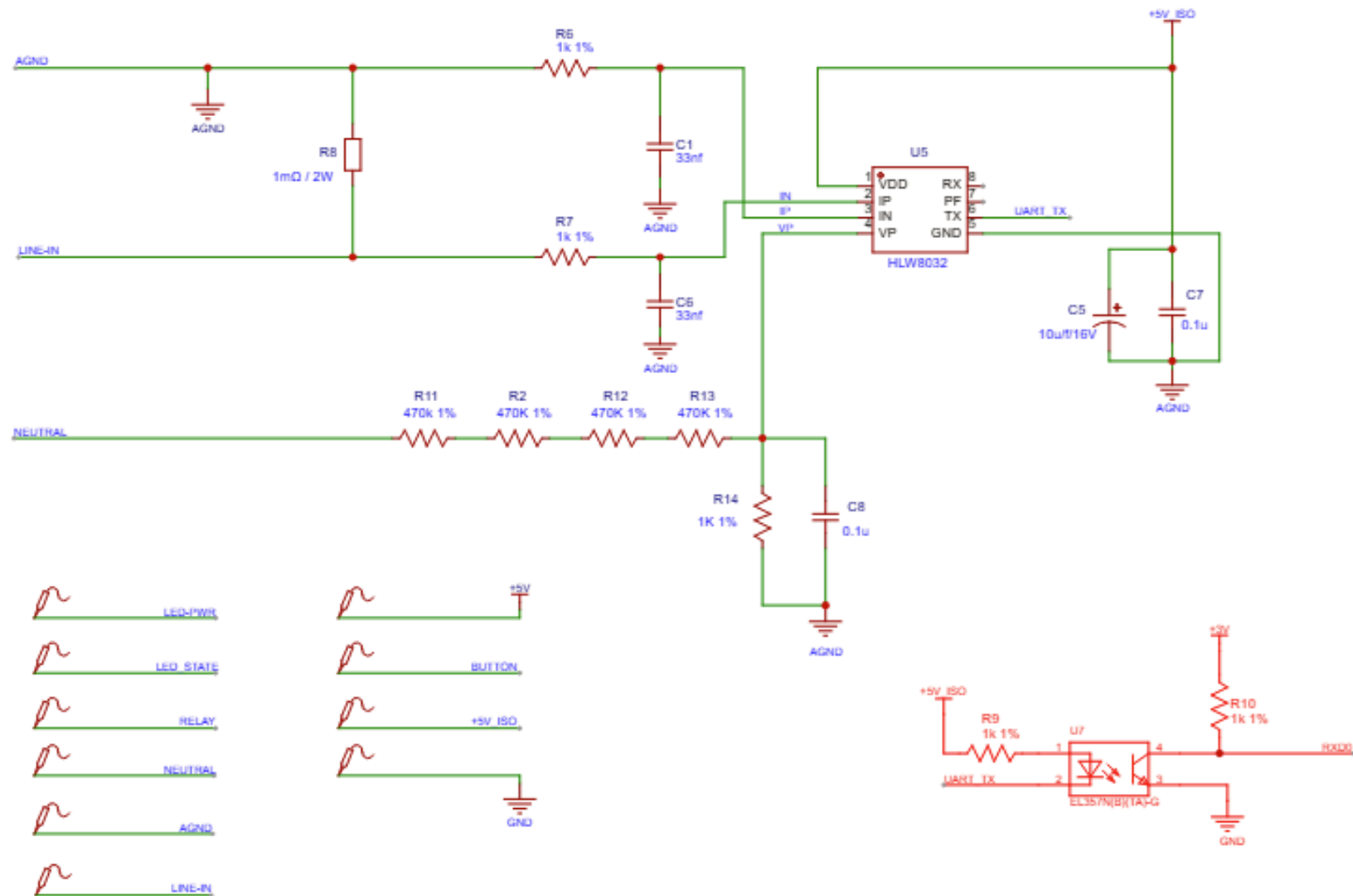
Schematic 1 / 3

GPIO
P1.10: LED
P1.14: Fake LED
P1.05: LED-PWR
P1.04: RELAY-S
P1.11: RXD0
P1.08: LED-STATE
P1.07: BUTTON
P1.06: Factory Reset



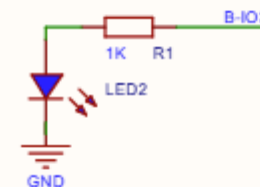
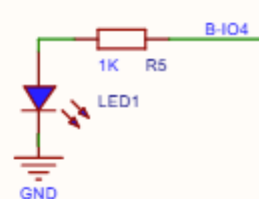
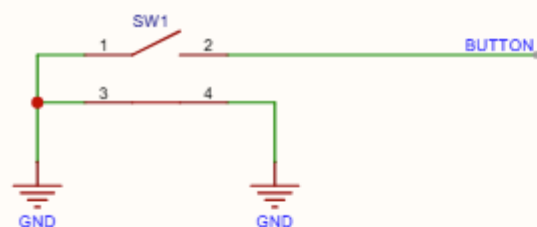
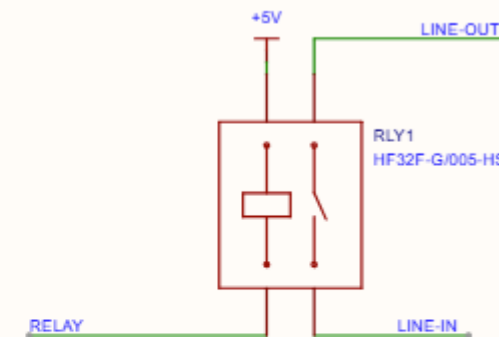
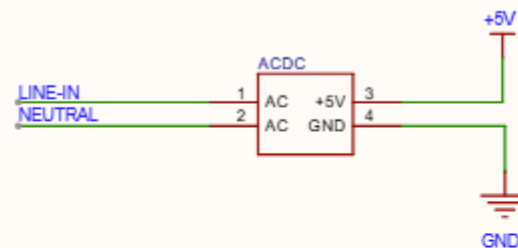
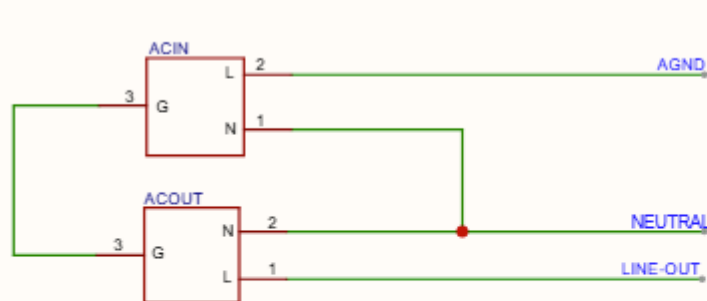
Smart Plug (HLW8032 metering)

Schematic 2 / 3



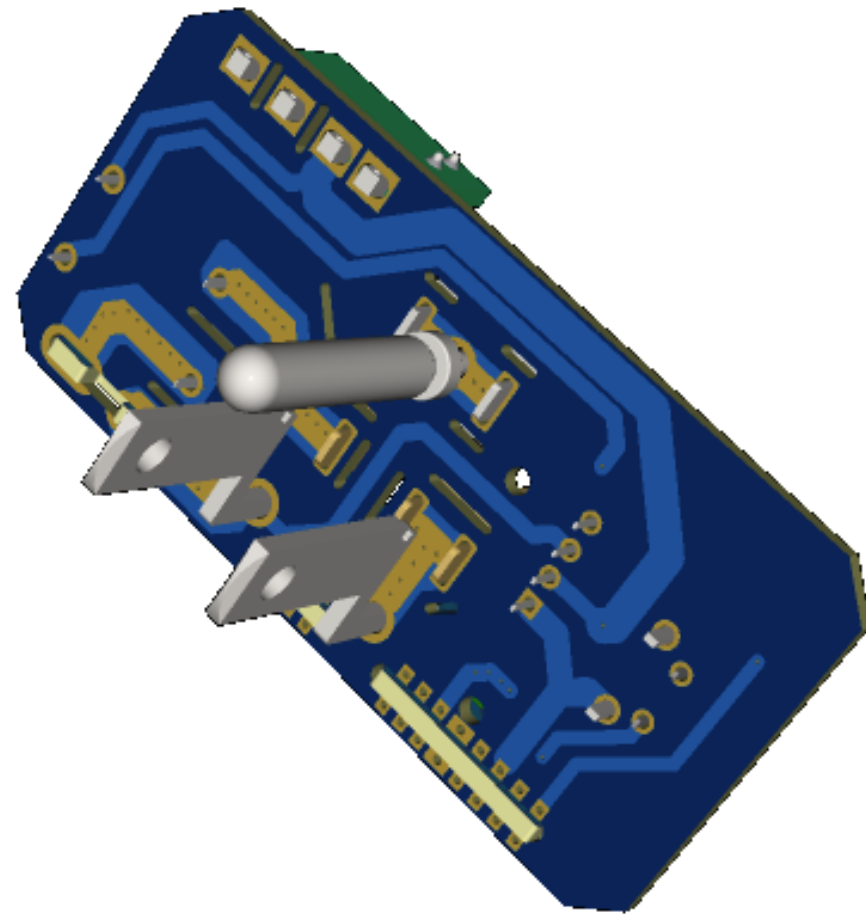
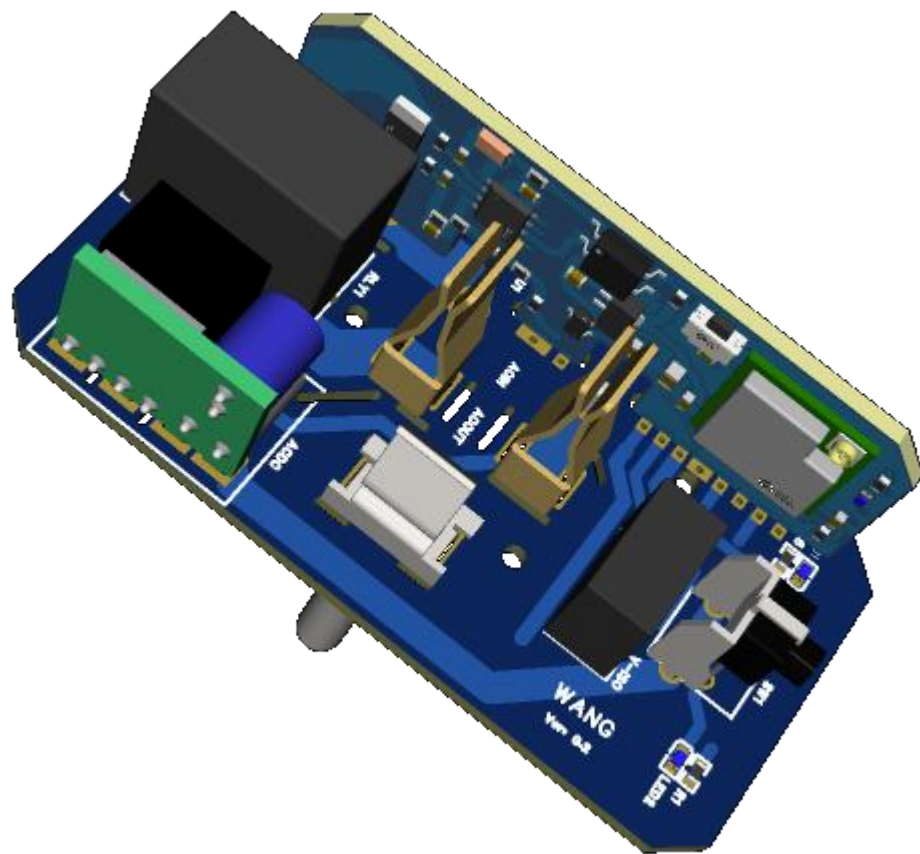
Smart Plug (HLW8032 metering)

Schematic 3 / 3



Garage Door Controller

PCBA-3D



PART 10

Lessons Learned

Every one of these cost us days.

This is the most valuable part of the deck.

LESSON 1

A stack overflow that looked like a dead device

SYMPTOM

Pairing hung at 'Connecting'. Device completely unresponsive. Needed several retries to pair at all.

ROOT CAUSE

The Bluetooth RX thread overflowed its 1200-byte stack during commissioning. On ARMv8-M that raises a UsageFault; Zephyr then HALTS the system — and debug builds do not reboot.

FIX

Raised CONFIG_BT_RX_STACK_SIZE to 2048. Found by halting the CPU over SWD and reading PSP == PSPLIM.

TAKEAWAY

When a device is 'dead', halt the core and read the registers. Do not guess.

LESSON 2

Six hours to come back after a power cut

SYMPTOM

After re-powering, a sensor reported nothing for hours, then recovered on its own. Restarting the border router also fixed it.

ROOT CAUSE

Matter sessions live in RAM and die with power. On boot the device retries subscription resumption on a Fibonacci backoff that CAPS AT SIX HOURS.

FIX

Overrode the resumption ceiling to 10 minutes in chip_project_config.h.

TAKEAWAY

'It fixes itself eventually' is a backoff timer, not luck. Go find it.

LESSON 3

An OTA that succeeded and changed nothing

SYMPTOM

Firmware uploaded over BLE DFU without error. Device still reported the old version afterwards.

ROOT CAUSE

We had added a project image-signing key. The bootloader ALREADY on the device only trusts the key it was built with, so it silently rejected the new image at boot.

FIX

Flash the new bootloader over SWD once. After that, OTA works normally.

TAKEAWAY

Changing the signing key breaks OTA for every deployed unit. Do it with the boards in your hand.

LESSON 4

A brand-new product that could not be paired

SYMPTOM

A new sensor failed commissioning immediately, with an attestation error. Older products on adjacent PIDs paired fine.

ROOT CAUSE

The Certification Declaration lists which PIDs it covers. The new product's folder carried an OLD copy of the CD that stopped below its PID.

FIX

Propagated the current CD to every product and added a consistency check.

TAKEAWAY

Duplicated files drift. If a file is copied per-product, something must verify the copies still agree.

LESSON 5

Two products became one device in Home Assistant

SYMPTOM

Home Assistant showed a single device carrying entities from two completely different products.

ROOT CAUSE

Both shipped the same hard-coded serial number, inherited by copy-paste. HA keys its device registry on the serial ALONE.

FIX

Derive the serial from the SoC's unique ID at runtime. Collisions become impossible.

TAKEAWAY

Any identity you type by hand will eventually be duplicated. Derive it instead.

LESSON 6

Power management silently disabling a sensor

SYMPTOM

A pressure sensor returned a frozen, impossible value while a second sensor on the SAME I2C bus worked perfectly.

ROOT CAUSE

Runtime power management suspends peripherals. The driver's suspend handler puts the sensor into sleep mode, where it stops converting — but still answers on I2C, so nothing looks broken.

FIX

Disable CONFIG_PM_DEVICE_RUNTIME for that product, or verify each driver's PM behaviour.

TAKEAWAY

A sensor that answers is not necessarily a sensor that measures.

LESSON 7

Counterfeit and defective sensors

SYMPTOM

Pressure read ~640 hPa instead of ~1010. Three separate boards, two re-soldered parts, identical behaviour.

ROOT CAUSE

The parts reported a valid chip ID and genuine calibration data — but their TEMPERATURE channel always returned zero. Pressure compensation depends on it, so the output was wrong.

FIX

Dump the raw registers. Confirmed by reading calibration and ADC values directly, then moved to a current, less-counterfeited part.

TAKEAWAY

EOL parts attract clones. Check the raw registers before blaming your board.

LESSON 8

Missing log lines are not evidence

SYMPTOM

A log line never appeared, so we concluded the code never ran. Twice. Both conclusions were wrong.

ROOT CAUSE

Deferred logging DROPS messages when the buffer overflows — which is exactly what happens during Matter's boot flood.

FIX

Quiet the noisy subsystems so the lines you care about survive, and verify with a second, independent method.

TAKEAWAY

Absence of a log is absence of evidence — not evidence of absence.

LESSON 9

A reset is not a power cycle

SYMPTOM

A sensor refused to initialise after every reflash, but worked once, inexplicably.

ROOT CAUSE

Debugger resets and reflashes restart the MCU — but NOT the peripherals on the board. Some parts latch their interface mode at their own power-up.

FIX

Physically remove power when testing anything sensor-related.

TAKEAWAY

'I reset it' and 'I power-cycled it' are completely different tests.

LESSON 10

The border router holds stale state

SYMPTOM

Devices stalled during commissioning, or went quiet after re-pairing. The device itself was provably healthy and idle.

ROOT CAUSE

The Thread border router kept stale entries for devices that had been erased and re-commissioned, and would not rebuild them on its own.

FIX

Restart the border router and allow a few minutes for the mesh to settle before concluding the device is at fault.

TAKEAWAY

Half of 'device' bugs live in the controller. Prove the device is idle first.

LESSON 11

Copy-paste is the biggest source of bugs

SYMPTOM

A product's build script regenerated a DIFFERENT product's files and reported success. Changes were silently discarded for months.

ROOT CAUSE

The script had been copied from a sibling project and every path still pointed at the original.

FIX

Derive per-product values from the folder name; never hard-code them in copied scripts.

TAKEAWAY

Every file you copy between products is a future bug. Generate, do not duplicate.

LESSON 12

Battery devices are a different animal

SYMPTOM

Timeouts, retries and recovery behaviour that worked on mains devices failed on battery ones.

ROOT CAUSE

A sleepy device only listens when it polls — up to minutes apart. Handshakes that assume prompt replies simply time out.

FIX

Test every network behaviour separately on battery hardware. Never assume mains results transfer.

TAKEAWAY

'It works on the plug' tells you nothing about the sensor.

LESSON 13

A copied formula turned maximum speed into 33%

SYMPTOM

Commanding gear 4 — the top setting — ran the fan at only a third of its speed.

ROOT CAUSE

The integration's original formula was $\text{gear} \times \text{percentage_step}$. On a 12-speed fan with 8.333% steps, $4 \times 8.333\% = 33\%$. That formula is only correct if the fan happens to have exactly 4 speeds too.

FIX

Convert proportionally instead: $\text{gear} \times 25\%$, so four gears land on 25 / 50 / 75 / 100%.

TAKEAWAY

Someone else's defaults were written for someone else's hardware. Re-derive any conversion against your own step count.

PART 11



The Debugging Toolkit

How to find out what is actually happening.

J-Link — the single most important tool here

- J-Link is SEGGER's debug probe, connected to the chip over two SWD wires
- It is four tools at once, which is why nothing else replaces it:
 - a flasher — writes firmware into RRAM
 - a debugger — breakpoints, halting the CPU, reading registers and memory
 - an RTT channel — log output without spending a single UART pin
 - a rescue tool — the only way in when the chip is locked or the code has run away
- Four wires: SWDIO, SWCLK, GND, and VTref to sense the target voltage

Only J-Link can get the truth out of a device that is completely unresponsive — the stack overflow in Lesson 1 was found by halting the CPU and reading registers.

J-Link in practice: commands and common states

```
nrftutil device list           # confirm the probe enumerated
nrftutil device program --firmware x.hex --x-family nrf54l
nrftutil device reset --x-family nrf54l
JLinkRTTLogger -Device nRF54L15_M33 -If SWD -Speed 4000 ...
```

What you see	What it really means, and what to do
debug port unavailable	the device is asleep – physically power-cycle it
No devices with requested S/N	the probe did not enumerate – replug the USB cable
reset did not restart the board	the RTT logger still holds SWD – close it first
RTT does not come back after power	JLinkRTTLogger does not reconnect – rerun it

We hit one of these four on almost every flashing session. Work through this table first — do not assume the firmware is broken because a tool threw an error.

RTT logging

- SEGGER RTT streams logs over the SWD debugger — no UART pins needed
- Deferred mode is fast but DROPS messages under load
- Immediate mode never drops, but formats on the caller's stack
 - It can itself cause a stack overflow — it did for us
- Quiet noisy subsystems so the lines you care about survive

```
CONFIG_USE_SEGGER_RTT=y  
CONFIG_LOG_BACKEND_RTT=y  
CONFIG_LOG_MODE_DEFERRED=y
```

Halt the CPU and read it

When a device is unresponsive and logging tells you nothing:

```
JLink.exe -CommandFile halt.jlink
  si SWD / speed 4000 / device nRF54L15_M33 / connect / halt / regs
mem32 0xE000ED28 4      # CFSR HFSR DFSR MMFAR
mem32 0xE000ED24 1      # SHCSR
```

Read this	It tells you
IPSR in XPSR	which exception is active (006 = UsageFault)
PSP == PSPLIM	stack overflow
SHCSR *ACT bits	a fault is currently active
PC via addr2line	exactly where it stopped

Scan the bus, dump the registers

- If a sensor 'does not work', first prove whether it answers at all
- An I2C scan over 0x08..0x77 tells you presence and address in one boot
 - Use a 1-byte READ — nRF TWIM cannot do zero-length transfers
 - A zero-length probe gave us a FALSE NEGATIVE and cost hours
- Then dump the device's own registers: ID, config, calibration, raw ADC

Reading raw registers ended a two-day investigation in one boot. We should have done it first, not last.

PART 12



Starting Your Own Product

A checklist that avoids everything above.

New product checklist

- Copy the closest existing product — then FIX every copied path and constant
- Allocate a new product code; derive PID, DAC prefix and discriminator from it
- Mint a per-product DAC rooted under a PAA your controller trusts
- Verify the CD covers your new PID
- Link the shared identity module so the serial is derived, never typed
- Define your hardware in the devicetree overlay AND enable its driver in Kconfig
- Regenerate the ZAP output after every data-model change
- Build debug first — with logging — and only then production
- Test on battery hardware separately if the product is battery powered
- Power-cycle, do not just reset, before believing any sensor result

Go build something.

Everything in this deck came from devices that are running in two homes today.

The mistakes are more useful than the successes — we hope they save you the time they cost us.